

DRONE BASED HOUSE BOUNDARY WALL MONITORING SYSTEM



By

MUHAMMAD TALHA	(2022-ARID-4127)
TEHREEM MALIK	(2022-ARID-4226)
TALHA IMRAN	(2022-ARID-4221)
HASSAN RAZA	(2022-ARID-4013)

**Bachelor of Science in Computer Science
(BSCS)**

**Barani Institute of Information Technology
PMAS Arid Agriculture University Rawalpindi**

Spring 2026

DRONE BASED HOUSE BOUNDARY WALL MONITORING SYSTEM



A report submitted in partial fulfillment of the requirements for the
degree of

Bachelor of Science in Computer Science

Submitted By

MUHAMMAD TALHA (2022-ARID-4127)

TEHREEM MALIK (2022-ARID-4226)

TALHA IMRAN (2022-ARID-4221)

HASSAN RAZA (2022-ARID-4013)

Supervised By

Mr. ALI BIN TAHIR

**Barani Institute of Information Technology
PMAS Arid Agriculture University Rawalpindi
Spring 2026**

CERTIFICATE

It is certified that the contents and form of thesis entitled "**Drone Based House Boundary Wall Monitoring System**" submitted by **Muhammad Talha, Talha Imran, Tehreem Malik, and Hassan Raza** have been found satisfactory for the requirement of the degree.

SUPERVISORY COMMITTEE

PROJECT SUPERVISOR:

Mr. Ali Bin Tahir

REPORT COORDINATOR:

Mr. Muhammad Khalid

WRITEUP COMMITTEE HEAD:

Ms. Noor ul Ain

ACKNOWLEDGEMENT

In The name of Almighty **ALLAH**, the Most Gracious and Most Merciful, who blessed us with the strength, knowledge, and patience to complete this project. All praise and thanksgiving are due to Him alone, for His guidance has been a constant source of encouragement throughout this journey. We would like to express our deepest gratitude to our supervisor **Mr. Ali Bin Tahir**, for his invaluable guidance, constructive feedback, and unwavering support throughout this project. His mentorship and positive criticism played a vital role in shaping this work. We are also sincerely thankful to our report coordinator Mr. Muhammad Khalid, and the write-up committee head Ms. Noor ul Ain, for their valuable suggestions and continuous guidance, which helped us refine and complete this work to the best of our ability. No words can fully express our gratitude to our group members, who stood by us at every stage of this project. Their dedication, cooperation, and tireless efforts made this journey not only successful but also enjoyable. Finally, and most importantly, we owe our deepest gratitude to our families, especially our parents, for their endless love, prayers, encouragement, and support during this entire degree program. Their belief in us has been our greatest strength, and without them, this achievement would not have been possible. May Allah bless and reward all of them abundantly.

ABSTRACT

“Drone Based House Boundary Wall Monitoring System” is an intelligent, low-cost, and GPS-independent perimeter security solution for residential and small commercial properties that combines three tightly integrated layers — an AI-powered surveillance backend, a custom laser-grid positioning network, and an autonomous patrol drone. The surveillance layer is built on a FastAPI backend with a PostgreSQL database and runs three computer-vision models in parallel across Entrance, Fence, and Insider cameras: a custom-trained YOLOv8l model, trained on 2,000+ annotated images for 60 epochs on a CUDA-enabled GPU, detects persons and weapons; a MediaPipe BlazeFace detector combined with the face_recognition library identifies authorized individuals at entrances; and a YOLOv8-pose model paired with a custom IoU-based tracker detects wall-climbing and unauthorized-zone intrusions along the boundary wall, with all detected events logged to the database and pushed instantly to the mobile application via WebSocket. The positioning layer avoids GPS and onboard vision-based localization entirely, relying instead on a custom 2D Cartesian laser grid — formed by intersecting KY-008 laser transmitter and receiver modules spanning a 13.8-metre area divided into a 5×7 cell grid — that provides real-time (X, Y) positional feedback through a network of five ESP32 microcontrollers (four slave nodes and one master node) communicating beam-break events via ESP-NOW to a backend laptop, which relays target-cell commands to the drone over a local Wi-Fi link. The drone itself is an F450-frame quadcopter equipped with a Pixhawk 6C Mini flight controller and a Raspberry Pi 4 companion computer that, using pymavlink, executes a strict finite-state machine to perform Manhattan-style, grid-aligned, non-diagonal navigation between cells, counting beam breaks to confirm arrival at each target location, with a manual failsafe providing a final safety layer, while the mobile application — implemented across Flutter, ReactJS, native Android, and React Native — provides a unified interface for live camera viewing, laser-grid visualization, alert management, authorized-person enrollment, and manual drone control.

TABLE OF CONTENTS

Contents	Page No.
CERTIFICATE	iii
ACKNOWLEDGEMENT	iv
ABSTRACT.....	v
TABLE OF CONTENTS.....	vi
LIST OF FIGURES	ix
LIST OF TABLES	x
LIST OF ANNEXURE	xi
CHAPTER 1 INTRODUCTION	1
1.1 Introductory Background.....	1
1.2 Problem Statement	2
1.3 Proposed Solution	3
1.4 Project Scope	3
1.5 Objectives	4
CHAPTER 2 BACKGROUND	6
2.1 Related Projects and Research	6
2.2 Challenges in Traditional Boundary Security Systems	7
2.3 The Proposed System.....	7
2.4 Key Features and Benefits	8
CHAPTER 3 CONCEPTUAL DESIGN	10
3.1 Requirement Elicitation	10
3.2 Requirement Specification.....	10
3.2.1 Functional Requirements	11
3.2.2 Non-Functional Requirements	12
3.2.3 Domain Requirements	12
3.3 Logical Design	13
3.4 Conceptual Diagram	13
3.5 Drone Hardware and Flight Control	14
3.5.1 Power Architecture (Separation of FC and Pi4 Supplies)	15
3.5.2 PVC Tracking Pole	15
3.6 Laser Grid and Power Architecture	16
3.6.1 Transmitter Side (Lasers).....	16

3.6.2 Receiver Side (ESP32 Laser Receiver Nodes)	16
3.6.3 Physical Grid Layout	16
3.7 Communication Pipeline (Real-Time Tracking)	17
3.7.1 Ground Network (ESP-NOW Beam-Break Detection)	17
3.7.2 Master ESP32 to Backend Laptop (USB Serial)	18
3.7.3 Air-to-Ground Network (Backend to Raspberry Pi 4).....	18
3.7.4 Latency Mitigation.....	18
3.8 Mathematical State Machine.....	19
3.8.1 Core States	19
3.8.2 Supporting / Placeholder States	20
3.8.3 Manhattan-Style Flight Routing	20
3.8.4 Grid Boundary and Tracking Failure Handling	22
3.9 Drone Navigation Backend.....	22
3.9.1 pymavlink vs. DroneKit.....	23
3.9.2 Manual Yaw Correction via RC	23
3.9.3 Integration with the AI Surveillance Backend.....	23
3.10 AI Surveillance Backend	23
3.10.1 Model 1 Custom Weapon and Person Detection (YOLOv8l)	24
3.10.2 Model 2 Face Detection (MediaPipe BlazeFace) and Face Recognition	24
3.10.3 Model 3 Wall-Climbing Detection (YOLOv8-Pose).....	25
3.10.4 Why a Custom IoU Tracker Instead of YOLO's Built-In Tracker	25
3.10.5 Cross-Camera Person Tracking and Alert Cooldown.....	25
3.10.6 Real-Time Alerting Pipeline	26
3.11 Frontend Applications.....	26
3.11.1 Common Frontend Architecture	27
3.11.2 Key Application Screens.....	27
3.12 Database Design.....	28
3.12.1 Key Table Relationships	29
3.13 Hardware Components Summary	30
CHAPTER 4 IMPLEMENTATION.....	37
4.1 Tools and Technologies	37
4.2 Pseudocode	39
4.2.1 Drone Manhattan-Routing State Machine (Raspberry Pi 4 / pymavlink).....	39
4.2.2 AI Surveillance Real-Time Alert Pipeline	39

4.3 Application Screens (GUI)	40
CHAPTER 5 CONCLUSION.....	55
5.1 Concluding Remarks.....	55
5.2 Future Direction	56
5.3 Limitations	56
REFERENCES	58
ANNEXURE.....	59

LIST OF FIGURES

Figure	Page No.
FIGURE 3.1: System Conceptual Diagram	14
FIGURE 3.2: Physical Laser Grid Layout (5×7 Cells, 13.8m Area)	17
FIGURE 3.3: ESP-NOW Communication Pipeline Diagram	18
FIGURE 3.4: Drone State Machine Diagram	21
FIGURE 3.5: Manhattan Routing Example (Cell 1,1 to Cell 4,5)	22
FIGURE 3.6: Real-Time Alert Pipeline (Detection Thread to WebSocket Broadcast)	26
FIGURE 3.7: Entity Relationship Diagram (Database Schema)	30
FIGURE 4.1: Splash Screen (A) Login Screen (B)	42
FIGURE 4.2: Sign Up Screen and Property List Screen	43
FIGURE 4.3: Add New Property Screen (A) Property List Screen (B)	44
FIGURE 4.4: Dashboard Screen	45
FIGURE 4.5: Manage Cameras Screen	46
FIGURE 4.6: Manage Drones Screen	47
FIGURE 4.7: Laser Grid (A) Grid Map Screen (B)	48
FIGURE 4.8: Fence Calibration Screen	49
FIGURE 4.9: Cell Calibration Screen	50
FIGURE 4.10: Alerts Screen	51
FIGURE 4.11: Alert Detail Screen	52
FIGURE 4.12: Authorized People Screen	53
FIGURE 4.13: Drone Control Screen	54
FIGURE 4.14: Alert Notifications on Mobile	55

LIST OF TABLES

Table	Page No.
TABLE 3.1: Drone Hardware Components.....	14
TABLE 3.2: Drone State Machine: Core States	19
TABLE 3.3: Drone State Machine: Supporting States	20
TABLE 3.4: Database Table Descriptions.....	28
TABLE 3.5: Complete Hardware Components Summary	35
TABLE 4.1: Tools and Technologies	37

LIST OF ANNEXURE

Annexure	Page No.
ANNEXURE.....	60
Annexure A: System Initialization.....	60
Annexure B: Mission Execution.....	60
Annexure C: Alert-Triggered Drone Dispatch	61
Annexure D: Emergency Protocols	62

CHAPTER 1

INTRODUCTION

“Drone Based House Boundary Wall Monitoring System” redefines residential perimeter security by fusing autonomous drone patrol, AI-powered threat detection, and a custom GPS-free navigation grid into a single, intelligent protection platform. At its core, the system deploys an F450-frame quadcopter — guided not by satellite signals, but by a precision-engineered 2D laser grid spanning 13.8 metres and divided into 35 virtual cells — that autonomously patrols the boundary wall and physically dispatches itself to the exact location of any detected threat. Three parallel computer-vision pipelines run continuously across Entrance, Fence, and Insider cameras: a custom-trained YOLOv8l model identifies persons and weapons, MediaPipe BlazeFace with face recognition distinguishes authorized individuals from intruders at the gate, and a YOLOv8-Pose tracker detects anyone attempting to climb the boundary wall. The moment a threat is confirmed, a real-time alert is pushed via WebSocket to the property owner's mobile application — built across Flutter, ReactJS, native Android, and React Native — and the drone is autonomously dispatched to the corresponding grid cell for immediate visual verification, all without a single GPS signal or human pilot. This project does not merely monitor a boundary — it actively defends it.

1.1 Introductory Background

Residential and small commercial properties have long depended on CCTV-based surveillance as their primary boundary security measure, yet these systems remain fundamentally passive tools — designed to record rather than respond. A camera positioned along a boundary wall will faithfully capture footage of an intruder climbing over it, but will take no action, raise no alert, and notify no one unless a human operator happens to be watching at that precise moment. This dependency on continuous human monitoring is impractical for the vast majority of households and small businesses, where dedicated security personnel are either unavailable or unaffordable. The natural next step — deploying autonomous patrol drones — introduces an equally serious challenge: virtually all consumer and commercial drone navigation systems depend on GPS positioning, and GPS performance degrades

significantly in exactly the environments where boundary patrol drones must operate. Tall boundary walls, dense residential construction, metallic roof structures, and narrow corridors all cause satellite signal multipath interference, making GPS-guided flight unreliable and potentially dangerous at low altitudes close to structures. Alternative navigation approaches such as SLAM and optical flow avoid the GPS dependency but replace it with expensive onboard hardware, high computational demands, and sensitivity to lighting and surface conditions that make them impractical for low-cost residential deployment. The result is a clear and unmet gap in the residential security landscape: properties need an automated, always-on surveillance system that can actively classify threats in real time and dispatch a physical response to the exact location of a detected event, without depending on GPS, without requiring a human operator, and without demanding enterprise-level hardware budgets.

1.2 Problem Statement

Existing residential boundary security solutions share a fundamental architectural flaw: they detect but do not respond. Three specific deficiencies define this gap. First, conventional CCTV systems are entirely reactive — they record footage of an intrusion after it has already begun but cannot classify the nature of the threat, cannot distinguish an unauthorized intruder from an authorized guard, cannot determine whether a weapon is present, and cannot trigger any automated physical response. Second, autonomous drones — the most promising candidate for a rapid physical response mechanism — cannot be reliably deployed for close-quarters boundary patrol because GPS signals are consistently degraded in wall corridors and near buildings, and the available GPS-free alternatives (SLAM, optical flow) are too computationally expensive and hardware-intensive for small-scale residential use. Third, even in systems where AI-based detection has been integrated, a unified pipeline connecting real-time detection, instant owner notification, and autonomous physical dispatch to the precise location of the detected event does not exist — threats are logged, but owners are not notified immediately, and no mechanism directs a drone to the exact cell where the intrusion occurred. There is therefore a clear need for an integrated platform that combines multi-model AI-based threat classification with a low-cost, GPS-independent drone positioning mechanism, closing the loop between detection and physical verification automatically, precisely, and without human intervention.

1.3 Proposed Solution

To address the three identified deficiencies, the “Drone Based House Boundary Wall Monitoring System” proposes a three-layer integrated architecture in which AI-powered surveillance, GPS-free drone positioning, and autonomous flight control operate as a single, unified platform. The first layer is an AI surveillance backend built on FastAPI and PostgreSQL, running three specialized computer-vision models in parallel across three camera types: a custom-trained YOLOv8l model — trained on 2,000+ annotated images for 60 epochs on a CUDA GPU — detects persons and weapons on Fence and Insider cameras; a MediaPipe BlazeFace detector combined with the `face_recognition` library identifies and verifies authorized individuals at Entrance cameras; and a YOLOv8-Pose model with a custom IoU-based tracker detects wall-climbing and unauthorized-zone intrusions, with all confirmed events immediately broadcast to the owner's mobile application via WebSocket. The second layer is the laser-grid positioning network — a physical 5×7 cell grid spanning 13.8 metres, formed by KY-008 laser transmitter and receiver modules mounted at exactly 1 metre height, monitored by four Slave ESP32 microcontrollers communicating beam-break events via ESP-NOW to a Master ESP32, which relays live (X, Y) coordinate data to the backend over USB serial — providing real-time drone positioning without any GPS dependency. The third layer is the drone itself — an F450-frame quadcopter with a Pixhawk 6C Mini flight controller and Raspberry Pi 4 companion computer running a pymavlink-based finite-state machine that performs Manhattan-style, grid-aligned navigation between cells by counting laser beam breaks — connected to the surveillance layer such that any critical alert automatically triggers an autonomous dispatch to the exact grid cell where the threat was detected.

1.4 Project Scope

The scope of this project covers the complete design and implementation of all three system layers along with four parallel mobile application implementations. On the surveillance side, the scope includes training and integrating the custom YOLOv8l weapon and person detection model, integrating MediaPipe BlazeFace and `face_recognition` for authorized-person verification, implementing the YOLOv8-Pose wall-climbing detector with custom IoU-based tracking and cell-level localization, and building the FastAPI backend with PostgreSQL

storage, JWT authentication, WebSocket-based real-time alerting, and a per-person 60-second alert cooldown mechanism. On the navigation side, the scope includes physically constructing the 5×7 laser grid across a 13.8-metre perimeter area, wiring and programming the five ESP32 microcontrollers for beam-break detection and ESP-NOW communication, and implementing the drone-dispatch backend that receives beam-break data over USB serial and relays flight commands to the Raspberry Pi 4 over a local Wi-Fi network. On the drone side, the scope includes assembling the F450 quadcopter with the specified propulsion, power, and sensor components, and implementing the pymavlink-based finite-state machine covering all seven core states — emergency kill, takeoff, land, yaw right, yaw left, pitch forward start, and pitch forward stop — for autonomous Manhattan-style grid navigation. On the application side, the scope includes building mobile interfaces across Flutter, ReactJS, native Android, and React Native, each providing camera management, laser-grid visualization, alert handling, authorized-person enrollment, and drone control. The project explicitly excludes GPS-based navigation of any kind, automated obstacle avoidance beyond grid boundaries, and automated recovery from grid-tracking failures, which are handled entirely by manual failsafe.

1.5 Objectives

The main objectives of this project are as follows:

- To design and implement a multi-model AI surveillance pipeline capable of detecting persons, weapons, unauthorized faces, and wall-climbing events in real time across Entrance, Fence, and Insider cameras.
- To train a custom YOLOv8l model on a manually annotated dataset of 2,000+ images for accurate person and weapon detection.
- To implement face detection and recognition using MediaPipe BlazeFace and the face_recognition library for distinguishing authorized individuals (Guard, Guest, Authorized Person) from unauthorized persons.
- To implement wall-climbing and insider-zone intrusion detection using YOLOv8-pose and a custom IoU-based person tracker, with cell-level location reporting.
- To design and construct a GPS-free, 2D Cartesian laser-grid positioning system using KY-008 laser modules and ESP32 microcontrollers communicating via ESP-NOW.

- To implement an autonomous, Manhattan-style drone navigation state machine using pymavlink on a Raspberry Pi 4 connected to a Pixhawk 6C Mini flight controller.
- To build a FastAPI backend that unifies surveillance alerts, the laser-grid position feed, and drone-dispatch commands, with PostgreSQL for persistent storage and WebSocket for real-time alert delivery.
- To develop multi-platform mobile applications (Flutter, ReactJS, Android, React Native) providing camera management, live streaming, laser-grid visualization, alert handling, and drone control.
- To provide a manual failsafe mechanism (emergency kill / emergency land) for handling grid-tracking failures or unexpected drone behavior.

CHAPTER 2

BACKGROUND

This chapter reviews existing commercial and research systems relevant to autonomous drone surveillance and boundary security, identifies the limitations of traditional approaches, and describes the proposed system's capabilities and key features.

2.1 Related Projects and Research

This project is titled as "**DRONE BASED HOUSE BOUNDARY WALL MONITORING SYSTEM**". This idea is unique in its nature as no similar study combining GPS-free laser-grid drone navigation with multi-model AI surveillance has been conducted in the past. It is a new project, developed by researchers Muhammad Talha, Talha Imran, Tehreem Malik, and Hassan Raza. In this project, the following functionalities are provided: property owners can monitor their boundary wall in real time using AI-powered cameras that detect wall-climbing, unauthorized persons, and weapons. The system also offers autonomous drone dispatch capabilities using a custom laser-grid positioning network, providing precise, GPS-free navigation for immediate visual verification of detected threats. For residential and small commercial properties, the application incorporates a multi-platform mobile interface (Flutter, ReactJS, Android, React Native), allowing seamless monitoring and drone control from any device. Additionally, users can enroll authorized persons with face photos for automatic recognition, fostering a more proactive and automated approach to boundary security.

This system is designed for residential and small commercial property owners who require continuous, automated perimeter monitoring without relying on GPS or constant human supervision. Property owners can monitor all detection activity through a mobile dashboard displaying live camera feeds, drone status, and real-time alerts. The system autonomously patrols the boundary using a physical laser grid and performs threat detection and drone dispatch without manual intervention. This platform is very helpful for property owners with no drone-piloting experience, as they can operate the entire system through a simple mobile interface.

A number of commercial and research systems address related aspects of autonomous surveillance and drone navigation. DJI's consumer and enterprise drones include obstacle

detection, path planning, and GPS-based autonomous flight, using AI-powered vision sensors for stability and mission execution. While powerful, these systems are GPS-dependent and cannot operate reliably in the close-quarters environment adjacent to a residential boundary wall, where satellite signal quality is poor. Skydio specializes in AI-powered autonomous drones that use deep learning and computer vision for obstacle avoidance and navigation, primarily for surveillance and inspection applications; however, their system requires unobstructed camera coverage for localization and is not designed for fixed-point perimeter patrol without GPS. Commercial smart home security products such as Ring and Arlo provide AI-based person detection at entrances but do not integrate drone-based response or boundary-wide intrusion detection, remaining passive recording systems without automated threat classification or physical response capability. Academic work such as UAV-DETR (Zhang, Liu, Gan, & Zhu, 2025) explores efficient end-to-end object detection for UAV imagery, informing the choice of YOLO-based architectures for this project, while studies such as Guna, Ganeriwala, & Bhattacharyya (2024) on machine learning engineering for object detection and tracking by UAVs informed the selection of pose-based tracking for wall-climbing detection.

2.2 Challenges in Traditional Boundary Security Systems

Traditional boundary security systems based purely on CCTV face several critical limitations. They require continuous human monitoring to be effective, and cannot distinguish between authorized and unauthorized individuals without manual review. They provide no automated classification of threats such as weapons, and offer no mechanism for physical, real-time response to a detected intrusion. Where drones have been proposed as a response mechanism, GPS-dependent navigation introduces reliability problems near boundary walls and buildings, where satellite signal quality is poor. Vision-based navigation alternatives require expensive onboard compute and are sensitive to lighting and surface conditions, making them impractical for low-cost residential deployment.

2.3 The Proposed System

The Drone Based House Boundary Wall Monitoring System overcomes these challenges by integrating real-time, multi-model computer vision with a custom GPS-free laser-grid positioning system. The system provides the following capabilities:

Multi-Camera Threat Classification: Entrance cameras run face detection and recognition to flag unauthorized individuals. Fence cameras run wall-climbing detection and weapon detection. Insider cameras run restricted-zone intrusion and weapon detection.

Real-Time Alerting: Detected events are immediately written to the database and pushed to the mobile application via WebSocket, with snapshots and short video clips attached.

GPS-Free Drone Dispatch: When an alert is associated with a specific grid cell, the drone can be autonomously routed to that cell using the laser-grid positioning system and Manhattan-style navigation.

Person Tracking with Cooldown: A cross-camera person-tracking service prevents duplicate alerts for the same individual within a 60-second window, using face-encoding similarity.

2.4 Key Features and Benefits

- **Custom-Trained Weapon and Person Detection:** A YOLOv8l model trained on 2,000+ manually annotated images across two classes (person, weapon) for 60 epochs on a CUDA GPU.
- **Face Detection and Recognition:** MediaPipe BlazeFace for fast face detection and the face_recognition library for 128-dimensional encoding and matching against authorized individuals.
- **Wall-Climbing and Insider Intrusion Detection:** YOLOv8-pose with 17-keypoint analysis and a custom IoU-based tracker to determine whether a person's body is inside a calibrated fence polygon.
- **Cell-Level Alert Localization:** Each camera's monitored area is subdivided into named cells (e.g., A1, A2), allowing alerts to specify exactly where an intrusion occurred.
- **GPS-Free Laser-Grid Navigation:** A 5×7 cell physical laser grid (13.8m) with ESP32-based beam-break detection provides real-time drone positioning without GPS.

- **Manhattan-Style Autonomous Routing:** The drone moves only along grid-aligned axes, using 90-degree yaw corrections and beam-break counting to navigate between cells.
- **Manual Failsafe:** Emergency kill and emergency land controls provide a manual override for any grid-tracking or communication failure.
- **Real-Time Push Notifications:** WebSocket-based alert broadcasting ensures the mobile application is updated instantly when an event is detected, without polling.
- **Multi-Platform Mobile Applications:** Independent implementations in Flutter, ReactJS, native Android, and React Native, each providing the full set of monitoring and control screens.

By combining automated AI-based threat classification with a physical, deterministic positioning grid, the system eliminates the need for constant human monitoring while enabling rapid, location-specific physical response via the patrol drone. The per-person cooldown mechanism reduces alert fatigue by preventing repeated notifications for the same individual within a short time window, while ensuring that genuinely new events are still reported immediately.

CHAPTER 3

CONCEPTUAL DESIGN

This chapter discusses the concepts, requirements, and overall data flow used in the design of the “**Drone Based House Boundary Wall Monitoring System**”. It covers how requirements were gathered, the functional and non-functional requirements of the system, and the conceptual diagram that illustrates how the AI surveillance layer, the laser-grid positioning layer, and the drone navigation layer interact. This chapter further presents the detailed implementation of the drone hardware and flight control configuration, the laser-grid positioning system, the real-time communication pipeline, the drone's mathematical state machine, the FastAPI-based drone navigation backend, the AI surveillance backend, the multi-platform frontend applications, the database design, and a complete hardware components summary.

3.1 Requirement Elicitation

The requirements for this project were gathered through a combination of supervisor consultation and team brainstorming sessions. In each meeting with our supervisor **Mr. Ali Bin Tahir**, the project's objectives and constraints were discussed and refined — particularly the strict requirement that the drone navigation system avoid GPS and traditional vision-based localization entirely, in favour of a custom physical laser-grid sensing approach. The team followed each instruction and recommendation from the supervisor throughout the design process. Additional requirements were derived from analysis of existing commercial perimeter security products and relevant academic literature on autonomous UAV navigation.

3.2 Requirement Specification

The requirement specification consolidates all requirements imposed on the design, implementation, and verification of the system. It covers the AI surveillance pipeline requirements, the laser-grid hardware and communication requirements, the drone state-machine requirements, and the mobile application requirements across all four platform implementations.

3.2.1 Functional Requirements

AI Surveillance Requirements:

- The system must detect persons and weapons using a custom-trained YOLOv8l model on Fence and Insider cameras.
- The system must detect faces at Entrance cameras using MediaPipe BlazeFace and compare them against authorized-person encodings using the face_recognition library.
- The system must detect wall-climbing and insider-zone intrusions using YOLOv8-pose with a custom IoU-based tracker.
- The system must apply a 60-second per-person cooldown to prevent duplicate alerts for the same individual.
- The system must store all detection events (face, weapon, alert) in PostgreSQL and broadcast new alerts to connected clients via WebSocket in real time.

Laser Grid and Drone Requirements:

- The system must track the drone's position using a 5×7 cell laser grid (13.8m area) without GPS.
- The system must detect laser beam breaks via ESP32 microcontrollers and relay them via ESP-NOW to a master ESP32.
- The master ESP32 must relay beam-break data to the backend via USB serial connection.
- The drone must navigate using Manhattan-style routing — moving only along grid-aligned axes with 90-degree yaw corrections.
- The drone must support the following states at minimum: emergency_kill, takeoff, land, yaw_right, yaw_left, pitch_forward_start, pitch_forward_stop.
- The system must provide a manual failsafe (emergency kill / emergency land) for grid-tracking or communication failures.

Mobile Application Requirements:

- Users must be able to register and log in using JWT-based authentication.

- Users must be able to create and configure properties, including the laser-grid dimensions (x_lasers, y_lasers, box width, box length, grid height).
- Users must be able to add, edit, and remove cameras (Entrance, Fence, Insider), assign them to grid cells, and calibrate fence polygons and detection cells.
- Users must be able to add, edit, and remove drones, including their home grid-cell location and MAVLink connection string.
- Users must be able to view live camera feeds (via MJPEG streaming) and live drone FPV video.
- Users must be able to view, filter, and act on alerts (mark as read, false positive, resolve, dispatch drone).
- Users must be able to enroll authorized people with face photos for recognition.

3.2.2 Non-Functional Requirements

- **Performance:** Detection pipelines must process video streams with frame-skipping (every 3rd or 5th frame) to maintain real-time responsiveness on available hardware. The drone's forward velocity must be kept low to compensate for communication latency in the beam-break pipeline.
- **Reliability:** Camera RTSP connections must automatically reconnect on stream loss. The drone must support a manual emergency failsafe at all times.
- **Availability:** All detection services (face, weapon, wall-climbing) must auto-start for all configured cameras when the backend server boots.
- **Usability:** The mobile application must present consistent screens and workflows across all four platform implementations.
- **Security:** All API endpoints and the WebSocket alert connection must require a valid JWT access token tied to the property owner's account.

3.2.3 Domain Requirements

- The laser grid shall be erected at exactly 1.0 metre above ground level with beams aligned to the X and Y axes of the property boundary.
- The drone operating altitude shall be maintained between 1.2 m and 1.5 m to ensure the PVC pole intersects the 1.0 m laser plane. Hard ceiling: 1.8 m.

- The PVC pole shall be 60–80 cm in length and suspended directly beneath the drone's centre of mass.
- The communication pipeline shall use ESP-NOW (ground network) and local Wi-Fi (air-to-ground) to minimise external infrastructure dependency.
- The 5th master ESP32 shall connect to the backend laptop via USB serial connection.
- The system must be compatible with Android (Flutter/React Native/native Android) and a ReactJS-based web interface.

3.3 Logical Design

The logical design consists of two cooperating subsystems. The first is the data-centric AI surveillance subsystem, where the FastAPI backend reads and writes structured records in PostgreSQL and pushes real-time updates via WebSocket. The second is the real-time positioning and control subsystem, where beam-break events and drone state transitions flow through the ESP32 network, the backend's serial interface, and the Raspberry Pi 4's pymavlink control loop. The two subsystems are connected at the point where a surveillance alert's grid cell becomes a drone-dispatch target.

Unlike a conventional reactive system, this project's logical design emphasizes an integrated alert-to-dispatch pipeline: when the AI surveillance layer creates a critical alert in a specific camera cell, that cell's coordinates are immediately forwarded to the drone navigation backend as the target destination. The drone then autonomously routes to that location for visual verification, closing the loop between detection and physical response.

3.4 Conceptual Diagram

The conceptual diagram below illustrates the integration of the three major components. The AI Surveillance Subsystem shows cameras feeding detection models, which write alerts to PostgreSQL and broadcast via WebSocket to mobile applications. The Laser Grid Positioning Subsystem shows the KY-008 laser grid, ESP32 slave/master nodes, and ESP-NOW and USB serial communication. The Drone Navigation Subsystem shows the Raspberry Pi 4 running the pymavlink state machine, communicating with the Pixhawk 6C Mini and with the backend over Wi-Fi. Key relationships: Cameras → AI Detection Models → AlertService

→ PostgreSQL + WebSocket → Mobile Applications; KY-008 Grid → Slave ESP32s → ESP-NOW → Master ESP32 → USB Serial → Backend; Backend → Wi-Fi → Raspberry Pi 4 → MAVLink → Pixhawk 6C Mini → Motors/ESCs.

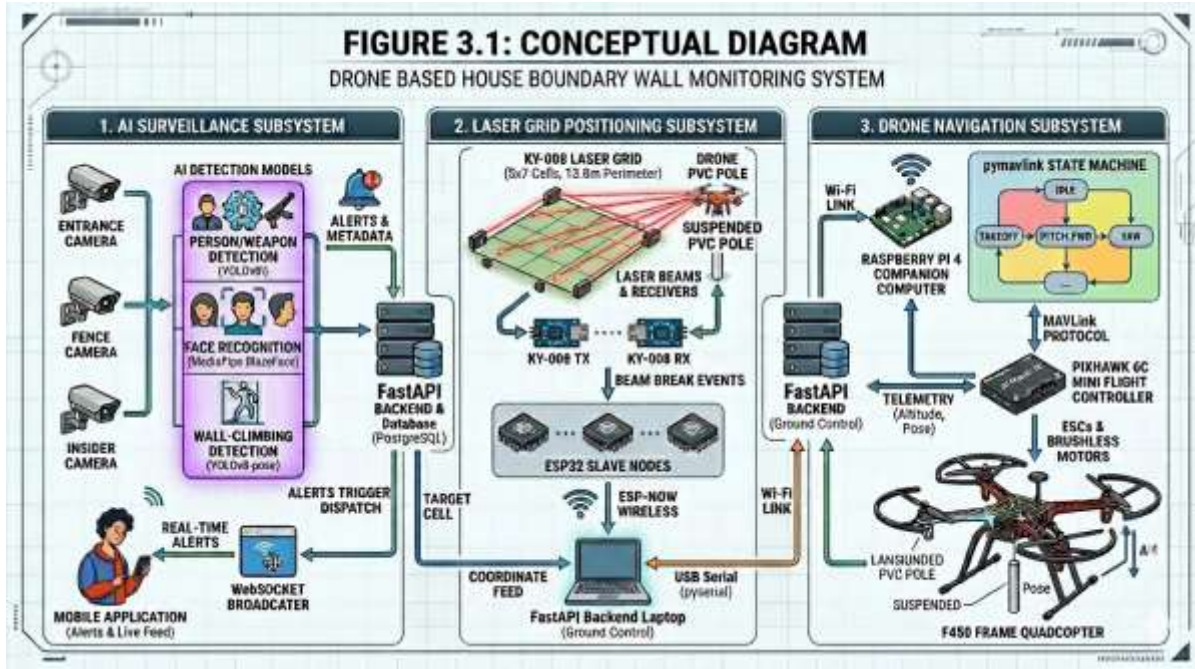


Figure 3.1: System Conceptual Diagram

3.5 Drone Hardware and Flight Control

The drone platform consists of a quadcopter frame fitted with a flight controller, companion computer, power distribution system, RC transmitter, and onboard sensors, configured as summarised in Table 3.1 below.

Table 3.1: Drone Hardware Components

Component	Specification
Frame and Propulsion	F450 frame, DJI 2212/920KV motors, E300 ESCs, 9450 propellers, prop guards, landing gear
Flight Controller	Pixhawk 6C Mini
Companion Computer	Raspberry Pi 4 (4GB)
Power Source	5200mAh 40C 4S LiPo battery (nominally 14.8V)

Component	Specification
Power Regulation	Two independent LM2596 buck converters — one for Flight Controller, one for Raspberry Pi 4 (each 5V output)
RC Transmitter	HotRC HT6A with PWM-to-PPM encoder
Heading Reference	Internal IMU and Compass (Pixhawk 6C Mini)
Altitude Sensing	VL53L1X time-of-flight sensor
Tracking Mechanism	Rigid PVC pole, 60–80cm, suspended below drone centre of mass

3.5.1 Power Architecture (Separation of FC and Pi4 Supplies)

A critical design decision in the power architecture is the use of two independent LM2596 buck converters — one dedicated exclusively to the Pixhawk 6C Mini flight controller, and one dedicated exclusively to the Raspberry Pi 4. Both convert the 5200mAh 40C 4S LiPo's output down to a regulated 5V rail, but on separate, isolated lines.

The Raspberry Pi 4 is a comparatively power-hungry companion computer, particularly under CPU load when executing the navigation state machine, communicating with the backend over Wi-Fi, and processing MAVLink messages. If the Pi 4 and the flight controller shared a single 5V regulator, a sudden current draw spike from the Pi 4 during heavy processing could cause a momentary voltage sag (brownout) on that shared rail. A brownout on the flight controller during flight is a serious safety hazard — it can cause loss of attitude control, an uncommanded disarm, or a crash. By isolating the two compute domains at the power-regulation stage, a power demand spike on the Pi 4 side cannot propagate to and destabilize the voltage supplied to the Pixhawk 6C Mini. This hardware-level separation of concerns directly mirrors the software-level separation in the system: the flight controller handles flight-critical attitude stabilisation, while the Raspberry Pi 4 handles higher-level navigation logic and communication — and neither can compromise the other.

3.5.2 PVC Tracking Pole

A rigid PVC pole of 60–80 cm length is suspended vertically from the drone's centre of mass. The pole hangs below the drone's airframe and intersects the horizontal laser beam plane

at 1.0 m above the ground as the drone flies at 1.2 m – 1.5 m altitude. The pole's tip breaks each laser beam sequentially, triggering the beam-break detection events that track the drone's (X, Y) grid position. This beam-breaking action is the sole sensing mechanism used for real-time position tracking — no onboard camera or GPS receiver is involved in this process.

3.6 Laser Grid and Power Architecture

The core navigation concept is a custom-built 2D Cartesian coordinate system implemented as a physical grid of laser beams, combined with a strict finite-state-machine controller. A strict design constraint of this project is that no GPS-based positioning and no traditional vision-based navigation may be used. All positional awareness for the drone is derived solely from the physical laser-grid sensor network described below.

3.6.1 Transmitter Side (Lasers)

The transmitter side of the laser grid consists of KY-008 laser diode modules. Power for the transmitter side is supplied by a 12V 3A adapter, stepped down to a parallel 5V lane that powers all KY-008 laser modules simultaneously.

3.6.2 Receiver Side (ESP32 Laser Receiver Nodes)

The receiver side is also powered by a 12V 3A adapter. It consists of four Slave ESP32 microcontrollers. Each Slave ESP32 has its own dedicated LM2596 buck converter, stepping the 12V supply down to 5V to power both the ESP32 itself and its two connected laser receiver modules. Each Slave ESP32 is therefore responsible for monitoring two laser beams.

3.6.3 Physical Grid Layout

The physical sensory grid is constructed at exactly 1 metre above the ground and spans a total monitored area of 13.8 metres, formed by two intersecting sets of KY-008 laser/receiver pairs:

- X-Axis lasers: 6 lasers running North-to-South, dividing the area into 5 columns.
- Y-Axis lasers: 8 lasers running East-to-West, dividing the area into 7 rows.

This produces a 5×7 grid, totalling 35 distinct virtual sectors (cells), covering the full 13.8-metre monitored area. The overall perimeter span is 2 metres per physical component length. Figure 3.2 illustrates the physical layout of the 5×7 laser grid.

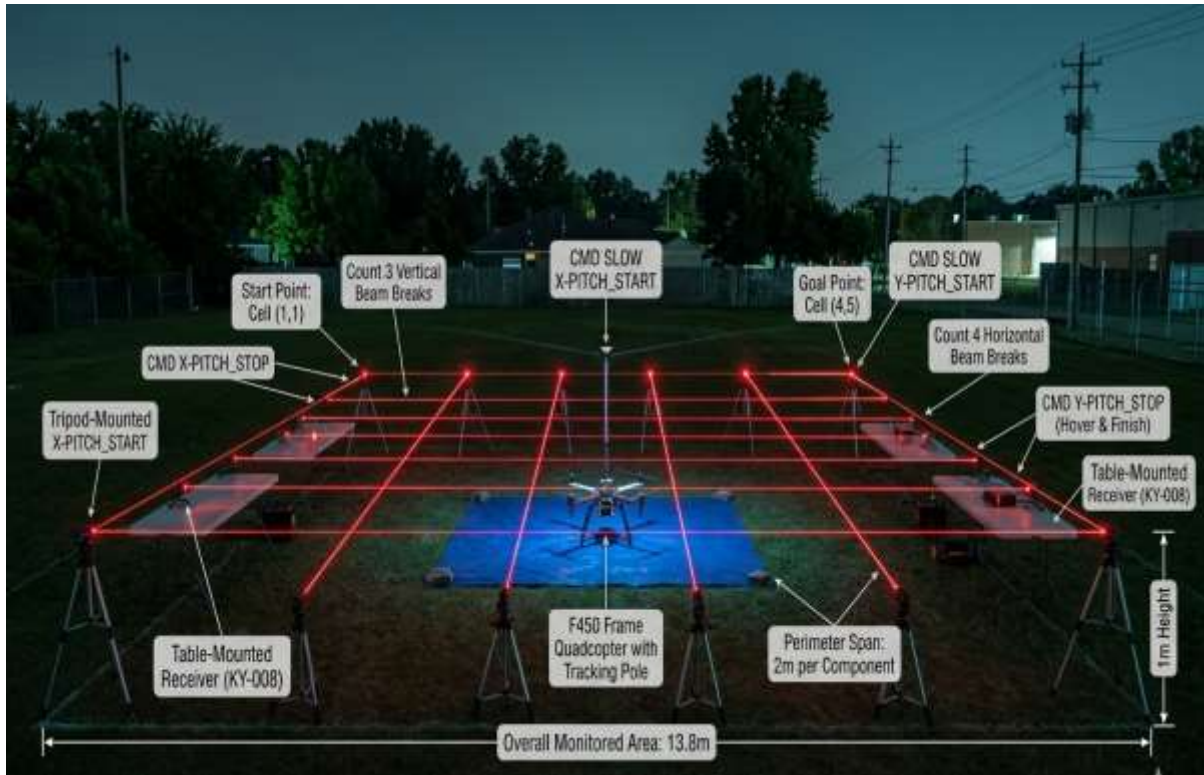


Figure 3.2: Physical Laser Grid Layout 5×7 Cells, 13.8m Area

3.7 Communication Pipeline (Real-Time Tracking)

Real-time positional awareness for the drone depends on a multi-stage communication pipeline that carries beam-break events from the physical laser grid to the Raspberry Pi 4 onboard the drone, and carries dispatch commands in the reverse direction.

3.7.1 Ground Network (ESP-NOW Beam-Break Detection)

The four Slave ESP32 microcontrollers continuously poll their connected receiver modules for beam interruptions. When the drone's tracking pole breaks a beam, the responsible Slave ESP32 instantly identifies which specific laser — X-axis or Y-axis, and which index along that axis — was interrupted. This beam-break event is transmitted via ESP-NOW, a low-latency, connectionless Wi-Fi protocol, directly to the fifth Master ESP32. ESP-NOW was selected because it provides direct, peer-to-peer communication between ESP32 devices without the overhead of establishing a traditional Wi-Fi network connection, minimizing latency between a beam break occurring and the event reaching the Master ESP32.

3.7.2 Master ESP32 to Backend Laptop (USB Serial)

The Master ESP32 is connected to the Backend Laptop via a USB Serial connection. Every beam-break event received from the four Slave ESP32s over ESP-NOW is forwarded by the Master ESP32 to the backend application over this serial link. The backend reads this serial stream using pyserial and aggregates incoming beam-break events into the drone's live (X, Y) cell coordinate, represented internally as a JSON array of coordinates.

3.7.3 Air-to-Ground Network (Backend to Raspberry Pi 4)

The Backend Laptop communicates with the Raspberry Pi 4 onboard the drone over a local Wi-Fi router network. Over this link, the backend sends the current target cell and any state-machine commands (e.g., emergency kill), and the Pi 4 reports its current state and tracked position back to the backend.

3.7.4 Latency Mitigation

The end-to-end path for a single beam-break event — laser interruption → Slave ESP32 → ESP-NOW → Master ESP32 → USB Serial → Backend Laptop → local Wi-Fi → Raspberry Pi 4 — introduces cumulative latency at each stage. To compensate, the drone is programmed to move at a deliberately slow, controlled forward velocity during the pitch_forward_start state. This ensures that even with accumulated communication latency, the drone does not overshoot a grid cell boundary before the corresponding beam-break event is registered and a stop command is issued.

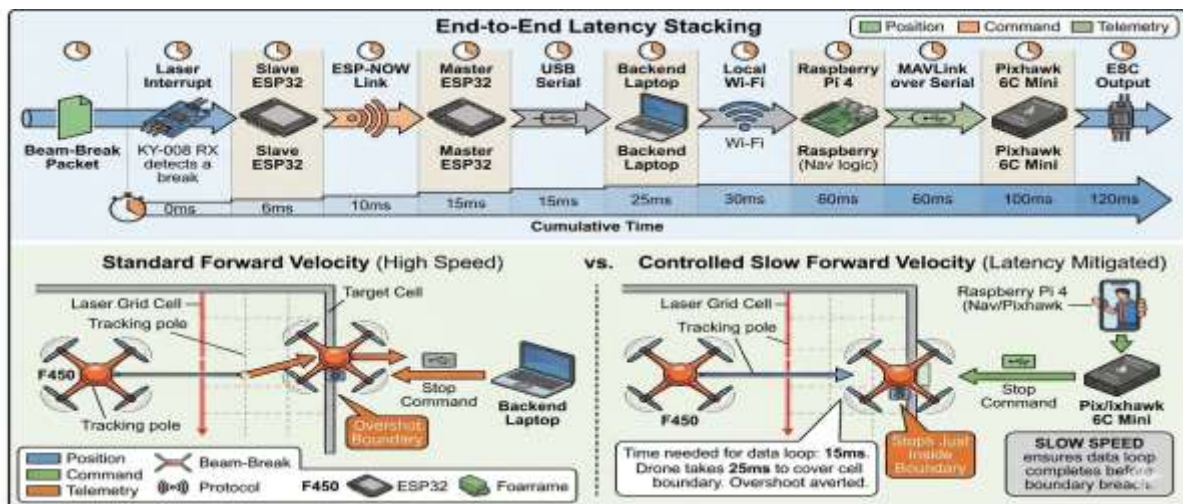


Figure 3.3: ESP-NOW Communication Pipeline Diagram

3.8 Mathematical State Machine

The drone's onboard control logic is implemented as a strict finite-state machine, executed on the Raspberry Pi 4 using pymavlink. Each state corresponds to a discrete MAVLink command or sequence of commands issued to the Pixhawk 6C Mini.

3.8.1 Core States

The following table defines the seven core states of the drone's finite-state machine, all of which are directly implemented in this project.

Table 3.2: Drone State Machine: Core States

State	Description
EMERGENCY_KILL	Immediately cuts motor output and forces disarm. This is the manual failsafe, triggered by the operator from any state at any time, overriding all other logic.
TAKEOFF	Commands the drone to ascend vertically to the operating altitude band (1.2m–1.5m, hard ceiling 1.8m) and hold position.
LAND	Commands a controlled vertical descent to ground level and disarm. Preferred over Emergency Kill when the drone is airborne.
YAW_RIGHT	Executes a 90-degree clockwise yaw rotation to realign the drone's heading with the next grid axis required for the current routing step.
YAW_LEFT	Executes a 90-degree counter-clockwise yaw rotation to realign the drone's heading with the next grid axis required for the current routing step.
PITCH_FORWARD_START	Commands a slow, constant forward velocity along the current heading axis. Remains active while beam breaks are counted toward the target cell distance.
PITCH_FORWARD_STOP	Commands zero forward velocity (full stop), transitioning the drone into a hover once the expected beam-break count for the current routing step is reached.

3.8.2 Supporting / Placeholder States

The following supporting states are included as structural placeholders within the finite-state machine, defining the surrounding control flow into which the seven core states above are embedded.

Table 3.3: Drone State Machine: Supporting States

State	Description
IDLE	Drone is armed/disarmed and stationary at its home cell, awaiting a dispatch command from the backend.
HOVER	Drone holds its current position and altitude with zero net velocity. Used between successive Manhattan-routing steps and as the terminal state of a dispatch.
AWAIT_BEAM_CONFIRM	Drone has issued PITCH_FORWARD_START and is waiting for the backend to confirm the expected beam-break count before transitioning to PITCH_FORWARD_STOP.
ALTITUDE_HOLD	A continuous background state, running in parallel with horizontal-motion states, that uses the VL53L1X sensor to maintain the commanded operating altitude.
RETURN_TO_HOME	The drone executes a reverse Manhattan-routing sequence back to its configured home grid cell.
FAULT	Entered if the communication pipeline times out or an unexpected beam-break sequence is received. Drone holds position and awaits manual operator input.

3.8.3 Manhattan-Style Flight Routing

Because the drone relies on the square laser grid for positioning, it does not fly diagonally. Instead, it uses a Manhattan routing protocol — analogous to a rook's movement in chess. As an example, to move the drone from Cell (1,1) to Cell (4,5), the onboard Raspberry Pi executes the following autonomous pymavlink sequence:

1. Command a slow, controlled forward velocity along the X-axis (PITCH_FORWARD_START).
2. Count exactly 3 vertical beam breaks to reach Column 4.

3. Command a full stop to zero velocity (PITCH_FORWARD_STOP).
4. Execute a 90-degree yaw to align with the Y-axis (YAW_RIGHT or YAW_LEFT, as appropriate).
5. Resume forward velocity (PITCH_FORWARD_START), counting exactly 4 horizontal beam breaks.
6. Execute a final stop and hover command upon arriving at Cell (4,5).

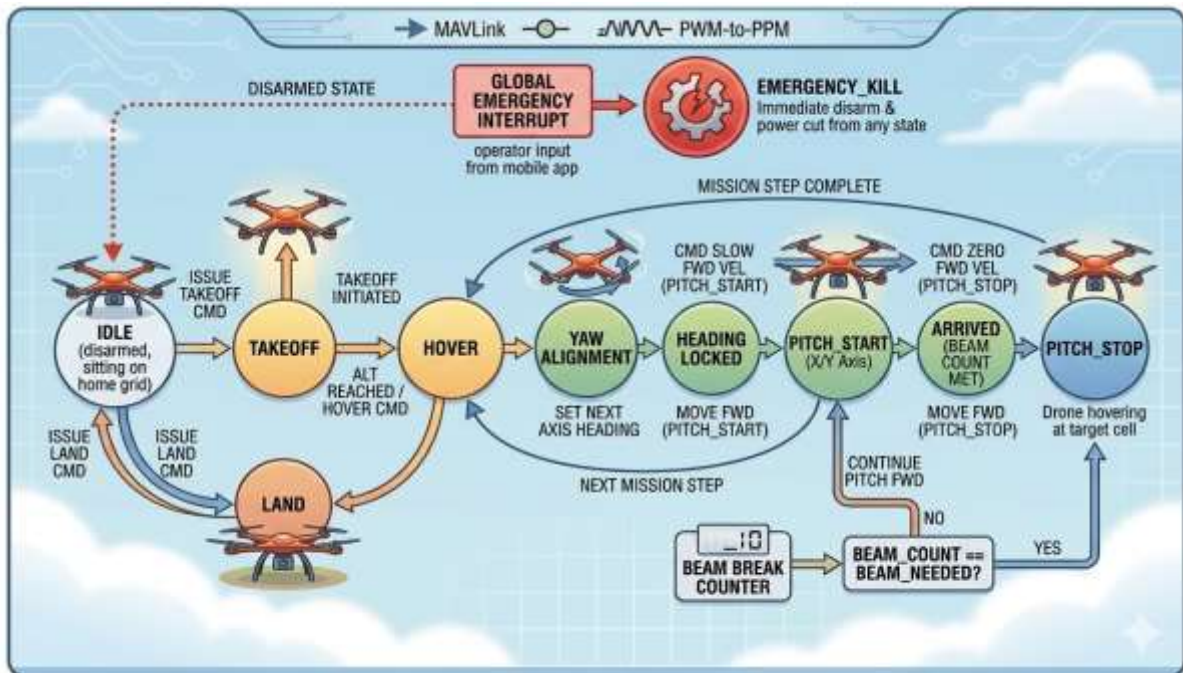


Figure 3.4: Drone State Machine Diagram

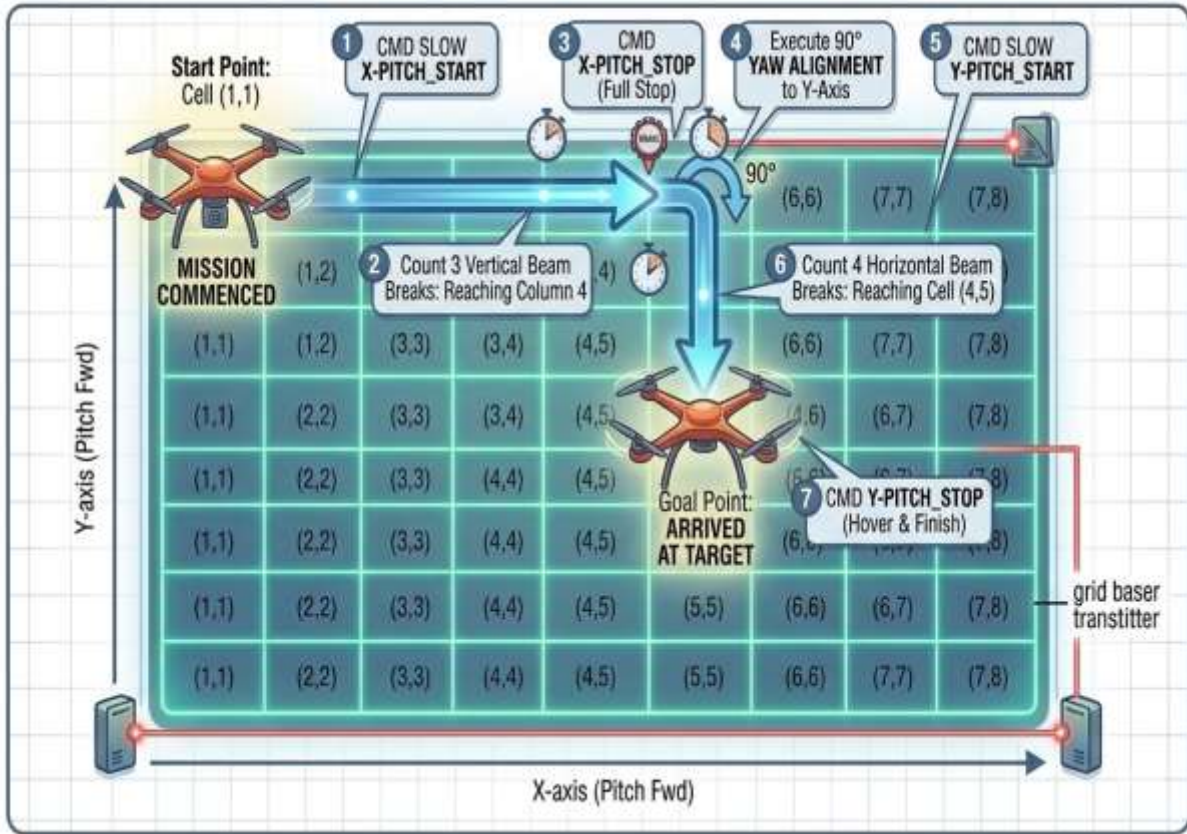


Figure 3.5: Manhattan Routing Example Cell (1,1) to Cell (4,5)

3.8.4 Grid Boundary and Tracking Failure Handling

The system does not implement an automated recovery algorithm for grid boundary errors or tracking failures. Instead, the designated failsafe is manual intervention: the operator presses the Emergency Land or Emergency Kill control in the frontend application, which is relayed to the Raspberry Pi 4 over the local Wi-Fi link and immediately interrupts the current state-machine execution via the EMERGENCY_KILL global interrupt.

3.9 Drone Navigation Backend

The drone navigation backend is a FastAPI service responsible for aggregating beam-break data from the Master ESP32, maintaining the drone's live grid position, accepting mission configuration (home cell and target cell) from the frontend application, and relaying state-machine commands to the Raspberry Pi 4 over the local Wi-Fi network.

3.9.1 pymavlink vs. DroneKit

The project uses pymavlink rather than the higher-level DroneKit library for drone communication. DroneKit provides convenient high-level abstractions but has been largely deprecated and offers limited control over the precise timing of velocity and yaw commands. pymavlink allows the control loop on the Raspberry Pi 4 to send raw MAVLink messages (such as SET_POSITION_TARGET_LOCAL_NED or velocity-set messages) and read telemetry directly. This level of control is necessary for the tight loop required by the Manhattan-routing state machine: start a forward velocity (PITCH_FORWARD_START), count a precise number of beam breaks, and issue a stop command (PITCH_FORWARD_STOP) at the correct moment.

3.9.2 Manual Yaw Correction via RC

Rather than relying on an automated vision- or magnetometer-only yaw correction loop, yaw alignment is corrected manually via the RC transmitter (HotRC HT6A). Before autonomous operation begins, the operator uses RC yaw trim to align the drone's heading precisely with the grid's X and Y axes. Once this baseline alignment is established, the onboard IMU/Compass maintains this reference heading during flight, and the YAW_LEFT / YAW_RIGHT states apply discrete 90-degree corrections relative to this manually-calibrated baseline.

3.9.3 Integration with the AI Surveillance Backend

The drone-dispatch backend operates as part of the same overall FastAPI system as the AI surveillance backend. When the surveillance layer's AlertService creates a critical alert — for example, 'Wall Climbing Detected in Cell A1' — the alert's associated grid cell is passed to the drone-dispatch backend as the target cell for a Manhattan-routing mission. This integration allows a detected boundary intrusion to automatically trigger a drone dispatch to the precise location of the event.

3.10 AI Surveillance Backend

The AI surveillance backend is a FastAPI application backed by a PostgreSQL database (via SQLAlchemy 2.0 ORM), providing JWT-based authentication, REST APIs for configuration and alert management, and a WebSocket endpoint for real-time alert delivery. It runs three independent AI detection pipelines across the property's cameras.

3.10.1 Model 1 Custom Weapon and Person Detection (YOLOv8l)

The first AI model is a custom-trained YOLOv8l (Ultralytics) object detector with two classes: 'person' and 'weapon'. The model was trained on a manually annotated dataset of over 2,000 images, for 60 epochs, on a CUDA-enabled GPU. This model runs on Fence and Insider cameras with a weapon confidence threshold of approximately 0.45. The detection loop runs inference on every 5th frame to balance accuracy and performance.

When a weapon is detected, the system searches for a nearby face within a proximity radius of 30% of the frame diagonal, to identify a potential 'holder' of the weapon. If a holder is identified and recognized as having the 'Guard' role (via the face recognition pipeline), the alert is suppressed and the event is logged only — this prevents false alarms for armed security personnel who are authorized to carry weapons. If the holder is a non-guard, unidentified, or no holder is found at all (a 'stray weapon'), a Weapon Alert is created. For every weapon detection event, the system saves a cropped image of the weapon, a full-frame snapshot, a face crop of the holder (if found), and a 5-second video clip of the event.

3.10.2 Model 2 Face Detection (MediaPipe BlazeFace) and Face Recognition

The second model combines MediaPipe's BlazeFace detector with the face_recognition library (built on dlib) for identity verification. It is used on Entrance cameras for primary face-based access monitoring, and within the weapon-detection pipeline to identify weapon holders. Face detection uses the MediaPipe Tasks API with the 'blaze_face_short_range.tflite' model and a minimum detection confidence between 0.5 and 0.6.

Detected face crops are passed to the face_recognition library, which computes a 128-dimensional face encoding for each detected face. The FaceRecognitionService compares this encoding against all stored face encodings for the property's authorized people, using Euclidean distance. A recognition threshold of 0.50 is used: if the Euclidean distance is below this threshold, the person is classified as 'Authorized'. If no match is found below the threshold, an 'Unauthorized Person at Entrance' alert is raised with high severity, including the face image, a full snapshot, and a 5-second video clip.

3.10.3 Model 3 Wall-Climbing Detection (YOLOv8-Pose)

The third model is a pre-trained YOLOv8-pose model (YOLOv8n-pose), producing 17 keypoints per detected person. It is used on Fence cameras to detect individuals climbing the boundary wall, and on Insider cameras to detect any person present in a restricted interior zone.

A 4-point polygon defining the fence/restricted-zone boundary is calibrated per camera via the mobile application. A detected person is flagged as an 'intruder' if at least 7 keypoints with confidence greater than 0.6 fall inside the calibrated polygon, and at least 2 of those keypoints are upper-body points (shoulders, elbows, or wrists). A custom IoU-based tracker (SimplePersonTracker) is used to maintain stable per-camera track IDs, with an IoU threshold of 0.3 and a maximum of 10 frames before a track is considered disappeared. A detection must be confirmed for 3 consecutive frames before an alert is raised, reducing false positives. Each tracked individual triggers at most one alert per session, implemented via a set of alerted track IDs.

3.10.4 Why a Custom IoU Tracker Instead of YOLO's Built-In Tracker

YOLO's built-in tracking (e.g., ByteTrack/BoT-SORT integrations) is a general-purpose solution that can be sensitive to occlusion and identity-switching in cluttered scenes. For this project's specific requirement — maintaining a stable identity for a person while they cross a fixed calibrated polygon, and ensuring each person triggers only one alert — a lightweight, custom IoU-based tracker provided more predictable and tunable behaviour. The custom SimplePersonTracker allowed direct control over the IoU threshold and disappearance tolerance, and integrated cleanly with the project's 'see-once' alerting and cell-based reporting requirements without the overhead of a full general-purpose tracking library.

3.10.5 Cross-Camera Person Tracking and Alert Cooldown

A central PersonTracker service matches detected faces across all cameras — Fence, Entrance, and Insider — using face-encoding distance with a face match threshold of 0.45. This service maintains a per-person alert cooldown of 60 seconds: if the same individual is detected again within this window, no duplicate alert is generated. Stale tracking records older than 24 hours are automatically cleaned up. This mechanism prevents the same person from generating repeated alerts as they remain in view across multiple cameras or frames, directly addressing the problem of 'alert fatigue' in surveillance systems.

3.10.6 Real-Time Alerting Pipeline

The end-to-end alerting flow proceeds as follows. A detection service detects an event on a dedicated background thread — one thread per camera, started automatically at server boot. The PersonTracker checks the 60-second per-person cooldown; if the person is within their cooldown window, no alert is created. If allowed, AlertService creates an Alert row in PostgreSQL with type, severity, confidence, camera info, grid cell, and snapshot/video URLs, and updates the property's active_alerts count. AlertService immediately broadcasts a JSON payload via the ConnectionManager to all WebSocket clients subscribed to that property. The mobile application receives the push instantly and updates the Alerts screen without polling. For critical severity alerts, the detected cell coordinate is optionally forwarded to the drone-dispatch backend to trigger an autonomous patrol response.

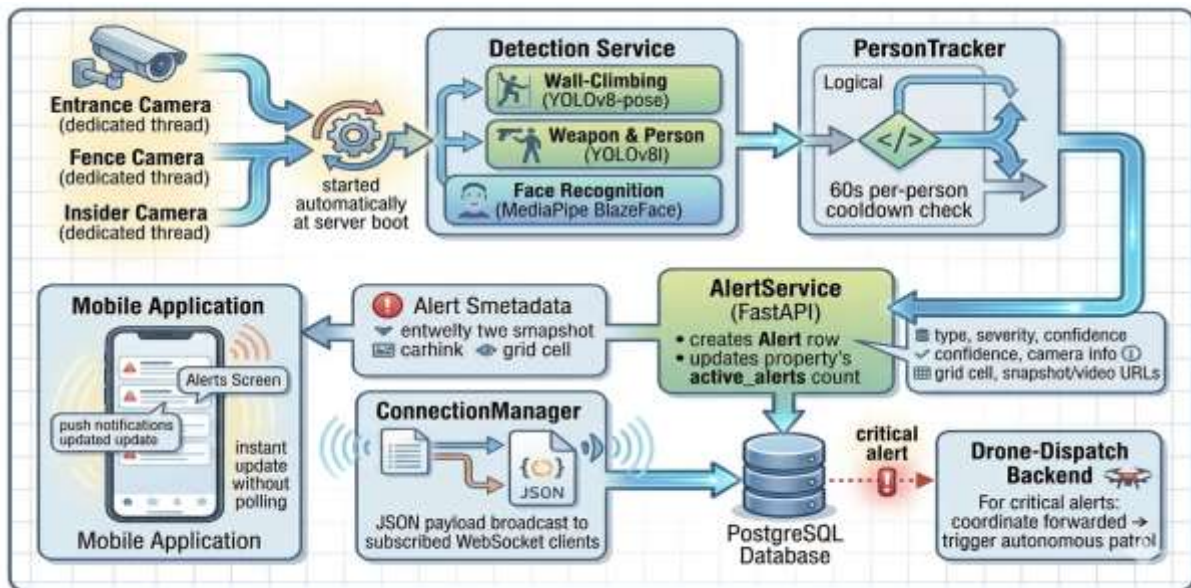


Figure 3.6: Real-Time Alert Pipeline Detection Thread to WebSocket Broadcast

3.11 Frontend Applications

The mobile and desktop frontend is implemented across four platform technologies — Flutter, ReactJS, native Android, and React Native — one primary implementation per team member, while sharing a common screen set and feature scope. Each implementation communicates with the same FastAPI backend over REST and WebSocket.

3.11.1 Common Frontend Architecture

Across all four implementations, the frontend follows a feature-based architecture. Authentication uses JWT-based login/signup, with the access token persisted in secure local storage and attached as a Bearer token to all subsequent API requests. Live camera streaming uses MJPEG-over-HTTP, which was selected because it requires no specialized media server or complex signaling, can be served directly by the FastAPI backend, and is straightforward to consume on all four target platforms using basic HTTP streaming and JPEG decoding. The stream auto-reconnects on error. Real-time alerts are received by subscribing to the WebSocket alerts endpoint, without polling. A shared warm editorial visual theme — off-white backgrounds, charcoal text, serif/sans-serif typography pairing, and sharp rectangular geometry — is applied consistently across implementations.

3.11.2 Key Application Screens

The following core screens are implemented consistently across all four platform variants:

- Splash, Login, and Signup screens — JWT authentication with email and password.
- Property List and Add/Edit Property — including Laser Grid Setup (X/Y laser counts, box width/length, grid height) and a map-based pin location picker.
- Dashboard — drone status, cameras online/total, active alerts, live camera and drone feed cards, WebSocket connection on load.
- Manage Cameras — list with type filter (All/Entrance/Insider/Fence), online/offline status, and grid cell assignment.
- Add/Edit Camera — name, RTSP/stream URL, camera zone type, grid cell location.
- Fence Calibration — draw 4-point boundary polygon over a live camera frame; Camera Cell Calibration — define named rectangular cells (e.g., A1/A2) within the polygon.
- Manage Drones and Add/Edit Drone — name, MAVLink connection string, home grid-cell location.
- Drone Control — live FPV video, grid position visualization, Return-To-Home and Land controls, directional/manual control pad.
- Grid Map / Laser Grid View — visualizes the property's laser grid as rows × columns with camera/drone/empty cell markers and a legend.

- Alerts — filterable list (New/All) and Alert Detail with event recording, severity, camera, time, confidence, Dispatch Drone / False Positive / Resolve actions.
- Authorized People — list, Add/Edit Person with Front/Left/Right face photo capture and 'Enroll Face' action.
- Flight Logs — drone name, flight type, takeoff/land time.
- Settings — navigation hub for Manage Cameras, Manage Drones, Authorized People, Drone Flight Logs, Laser Grid View, Manage Property Details, and Logout.

3.12 Database Design

The system uses a PostgreSQL database, accessed through SQLAlchemy 2.0 ORM models, to persist all configuration and event data. The following table summarises the thirteen database tables and their key fields.

Table 3.4: Database Table Descriptions

Table	Description and Key Columns
users	id, full_name, email (unique), hashed_password, is_active, created_at, updated_at. One user can own multiple properties.
properties	id, user_id (FK), name, address, latitude, longitude, x_lasers, y_lasers, box_width, box_length, grid_height, cameras_online, cameras_total, drone_status, active_alerts, created_at, updated_at.
cameras	id, property_id (FK), name, rtsp_url, camera_type (entrance/fence/insider), grid_cell_row, grid_cell_col, created_at, updated_at. One-to-one with fence_configs.
drones	id, property_id (FK), name, connection_string (MAVLink), status (Offline/Docked/Ready/Flying), home_cell_row, home_cell_col, created_at, updated_at.
flight_logs	id, property_id (FK), drone_id (FK), drone_name, flight_type (DISPATCH/PATROL/TEST), takeoff_time, land_time, created_at.
authorized_people	id, property_id (FK), name, role (Guard/Guest/Authorized Person), photo_urls (JSON array), face_encodings (JSON array of 128-d vectors), created_at, updated_at.

Table	Description and Key Columns
alerts	id, property_id (FK), camera_id (FK), alert_type, camera_name, severity (low/medium/high/critical), confidence, detected_cell, camera_cell_row, camera_cell_col, image_url, clip_url, is_read, status (active/resolved/false_positive), timestamp, created_at, resolved_at.
fence_configs	id, camera_id (FK, one-to-one), polygon_points (JSON — 4-point boundary, normalized 0–1 coords), cells_config (JSON), is_active.
fence_cells	id, camera_id (FK), cell_name (e.g., 'A1'), row, col, polygon_points (JSON — 4-point cell polygon). Used to subdivide the fence/insider area into named detection zones.
person_alert_tracking	id, tracking_id (UUID), property_id (FK), face_encoding (JSON), camera_ids_seen (JSON), last_camera_id (FK), last_alert_time, total_alerts_generated, total_detections, first_seen_at, last_seen_at, matched_person_id (FK), is_authorized. Central cross-camera identity and cooldown table.
face_detection_events	id, property_id (FK), camera_id (FK), is_authorized, matched_person_id (FK), matched_person_name, matched_role, recognition_confidence, tracking_id, face_image_url, snapshot_url, video_clip_url, alert_id (FK), detected_at, created_at.
weapon_detection_events	id, property_id (FK), camera_id (FK), weapon_class, weapon_confidence, weapon_bbox (JSON), holder_identified, holder_person_id (FK), holder_person_name, holder_role, is_guard, is_stray_weapon, tracking_id, snapshot_url, weapon_crop_url, face_image_url, video_clip_url, alert_id (FK), drone_triggered, detected_at, created_at.
alembic_version	version_num — Alembic migration tracking table.

3.12.1 Key Table Relationships

- users (1) — (N) properties: One user may own multiple properties.
- properties (1) — (N) cameras, drones, alerts, authorized_people, flight_logs, person_alert_tracking, face_detection_events, weapon_detection_events.

- cameras (1) — (1) fence_configs (cascade delete); cameras (1) — (N) fence_cells, alerts, face_detection_events, weapon_detection_events.
- drones (1) — (N) flight_logs.
- alerts (1) — (1, optional) weapon_detection_events and face_detection_events via alert_id.
- authorized_people (1) — (N) person_alert_tracking via matched_person_id; (1) — (N) weapon_detection_events via holder_person_id.

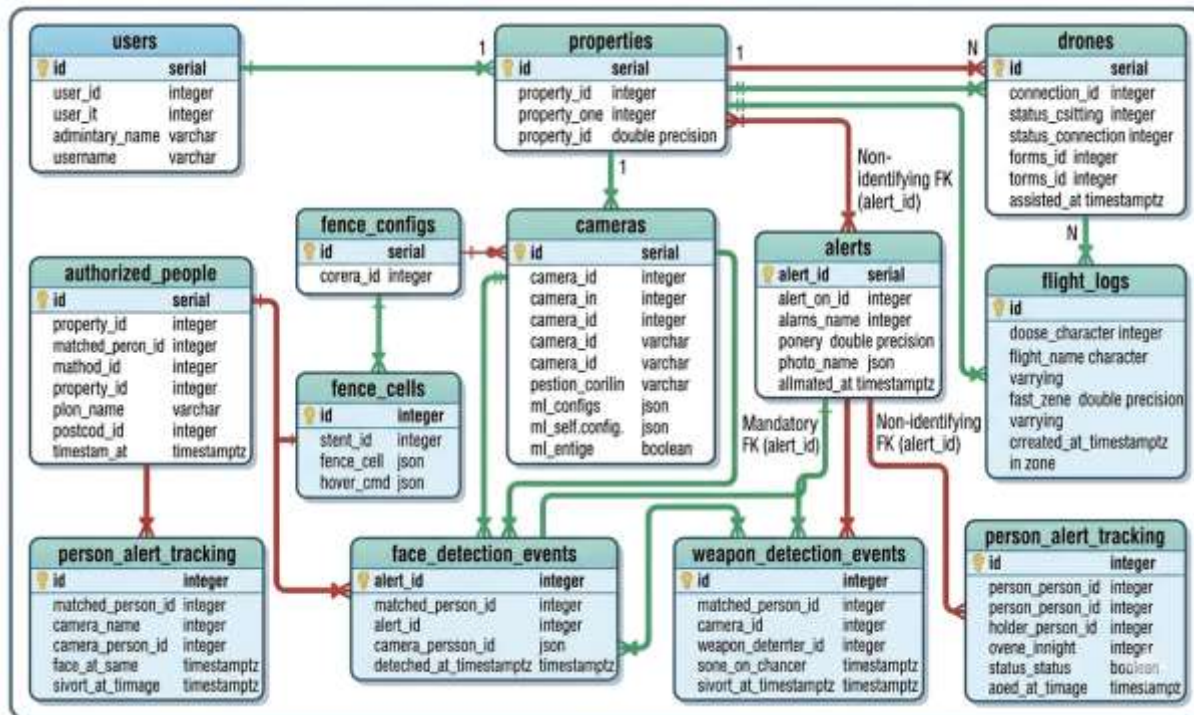


Figure 3.7: Entity Relationship Diagram (Database Schema)

3.13 Hardware Components Summary

The Drone-Based House Boundary Wall Monitoring System consists of several key hardware components, each contributing to navigation, stability, positioning, and autonomous flight execution. These components work together across the drone platform and the laser-grid sensing network to ensure efficient real-time operation and safe flight.

1- Flight Controller

The Pixhawk 6C Mini is the primary flight controller that manages all aspects of the drone's movement. It processes sensor inputs, executes pymavlink-based flight commands issued by the Raspberry Pi 4, and ensures stability during flight by continuously adjusting motor speeds through the E300 ESCs. It also receives manual override commands from the HotRC HT6A RC transmitter as an emergency failsafe.



2- Drone Frame

The F450 frame provides a sturdy yet lightweight structure for mounting all essential drone components, including motors, ESCs, the Pixhawk 6C Mini flight controller, the Raspberry Pi 4 companion computer, and the power system. Prop guards and landing gear are installed for safe low-altitude boundary-wall patrol. This frame ensures durability and stability during flight.



3- Companion Computer

The Raspberry Pi 4 (4GB) acts as the onboard companion computer, running the Python-based pymavlink state machine that drives the drone's Manhattan-style autonomous navigation. It receives target cell coordinates from the backend over Wi-Fi, counts beam-break events to track position, and issues MAVLink commands to the Pixhawk 6C Mini. It is powered by a dedicated LM2596 regulator, isolated from the flight controller supply to prevent brownout.



4- Camera (Pi Zero W Camera)

The Pi Zero W Camera module is used for real-time visual monitoring and FPV (First Person View) feed during drone dispatch. It enables the operator to visually verify the location of a detected boundary intrusion after the drone has been dispatched to the corresponding grid cell. The live video feed is streamed back to the mobile application for situational awareness.



5- RC Transmitter (HotRC HT6A 2.4GHz)

The HotRC HT6A 6-channel 2.4GHz RC transmitter provides manual control and serves as the primary operator failsafe. Connected to the Pixhawk 6C Mini via a PWM-to-PPM encoder, it allows the pilot to override autonomous flight at any time for emergency land, emergency kill, or manual yaw correction. It sends PWM signals controlling throttle, yaw, pitch, and roll — essential for pre-flight heading alignment with the laser grid.



6- Propellers (9045)

Four 9045 propellers (9-inch diameter, 4.5-inch pitch) are mounted on the DJI 2212/920KV brushless motors. This propeller size is matched to the 920KV motor winding and 4S battery voltage to provide adequate thrust for stable, controlled low-speed flight during grid traversal. Prop guards are fitted around each propeller for safe operation in proximity to the property boundary wall.



7- Motors (DJI 2212/920KV)

Four DJI 2212/920KV brushless motors provide the primary propulsion for the F450 quadcopter. The 920KV winding offers a balanced combination of thrust and efficiency at 4S (14.8V) voltage. These motors are controlled by the DJI E300 ESCs, which receive throttle commands from the Pixhawk 6C Mini's PID stabilization loops to maintain stable, precise hover during beam-break counting and grid traversal.



8- ESCs (DJI E300)

Four DJI E300 Electronic Speed Controllers (ESCs) regulate the power delivered to each of the DJI 2212/920KV brushless motors. They receive PWM signals from the Pixhawk 6C Mini and convert them into motor drive currents, providing smooth, responsive throttle control for the drone's attitude stabilization. The E300 system is optimized for multi-rotor use, ensuring consistent motor response across all four arms of the F450 frame.



9- Height Sensor (VL53L1X)

The VL53L1X time-of-flight sensor provides real-time altitude measurements, enabling the drone to maintain its operating altitude band of 1.2m to 1.5m throughout autonomous flight. This altitude ensures that the PVC tracking pole suspended below the drone consistently intersects the 1-metre laser plane for reliable beam-break detection. The sensor communicates via I2C and feeds the



ALTITUDE_HOLD background state of the navigation state machine.

10- Battery (5200mAh 4S LiPo)

The 5200mAh 40C 4S (14.8V) LiPo battery provides the main power source for the entire drone platform. It powers the four DJI 2212/920KV motors via the E300 ESCs, and feeds two independent LM2596 buck converters supplying regulated 5V to the Pixhawk 6C Mini and the Raspberry Pi 4 separately. The 40C discharge rating ensures adequate current delivery for sustained motor loads without causing voltage instability on the main power bus.



11- Power Regulators (LM2596 Buck Converters)

Two independent LM2596 DC-DC buck converters step down the 4S LiPo voltage to a regulated 5V — one dedicated to the Pixhawk 6C Mini and one to the Raspberry Pi 4. This electrical isolation ensures that current spikes from the Pi 4 under processing load cannot cause a brownout on the flight controller's supply rail, which would be a critical safety hazard mid-flight. Four additional LM2596 units power the Slave ESP32 receiver nodes on the laser grid.



12- Laser Grid & ESP32 Network

The laser grid consists of 14 KY-008 laser transmitter/receiver pairs (6 on the X-axis, 8 on the Y-axis), forming a 5×7 cell grid of 35 sectors across 13.8 metres at 1 metre above ground. Four Slave ESP32 microcontrollers each monitor two receivers and relay beam-break events to a Master ESP32 via ESP-NOW. The Master ESP32 forwards all events to the Backend Laptop via USB Serial for real-time (X, Y) position computation.



13- Drone



Table 3.5: Complete Hardware Components Summary

Component	Specification / Quantity
Drone Frame	F450 frame with prop guards and landing gear
Motors	DJI 2212/920KV brushless (×4)
ESCs	DJI E300 (×4)
Propellers	9450 propellers (×4)
Flight Controller	Pixhawk 6C Mini
Companion Computer	Raspberry Pi 4 (4GB)
Battery	5200mAh 40C 4S LiPo
Power Regulators	2× LM2596 buck converters — independent 5V rails for FC and Pi 4
RC Transmitter	HotRC HT6A with PWM-to-PPM encoder
Altitude Sensor	VL53L1X time-of-flight sensor
Tracking Pole	Rigid PVC pole, 60–80cm, suspended below drone centre of mass
Laser Transmitters	KY-008 laser diode modules — X-axis: 6, Y-axis: 8 (total: 14)
Laser Receivers	14 receiver modules (paired with KY-008 transmitters)
TX Power Supply	12V 3A adapter, stepped down to shared 5V lane for all lasers
RX Power Supply	12V 3A adapter, distributed via 4× dedicated LM2596 converters
Slave Microcontrollers	4× ESP32 (each monitoring 2 laser receivers)
Master Microcontroller	1× ESP32 (USB Serial to Backend Laptop)
Slave-to-Master Link	ESP-NOW (Wi-Fi peer-to-peer, connectionless, 2.4GHz)
Master-to-Backend Link	USB Serial connection
Backend-to-Drone Link	Local Wi-Fi router network
Grid Coverage Area	13.8 metres (5×7 cell layout, 35 cells total)
Laser Plane Height	1.0 metre above ground level

CHAPTER 4 IMPLEMENTATION

This chapter presents the full implementation details of **the Drone Based House Boundary Wall Monitoring System**, covering the development tools and runtime technologies used in the system, pseudocode for the two central control loops, and the complete set of application screens (GUI) implemented across the mobile platforms.

4.1 Tools and Technologies

Table 4.1 summarises the major tools, frameworks, and technologies used across the three layers of the system.

Table 4.1: Tools and Technologies

Component	Specification / Technology
Drone Flight Controller	Pixhawk 6C Mini
Companion Computer	Raspberry Pi 4 (4GB RAM)
Drone Control Library	pymavlink (MAVLink protocol)
Laser-Grid Microcontrollers	ESP32 — 4 Slave nodes + 1 Master node
Inter-MCU Communication	ESP-NOW (2.4GHz peer-to-peer, connectionless)
Master ESP32 to Backend	USB Serial connection (pyserial)
Backend to Drone (Pi4)	Local Wi-Fi router network
Backend Framework	FastAPI (Python 3.11)
Database	PostgreSQL with SQLAlchemy 2.0 ORM
DB Migrations	Alembic
Authentication	JWT (python-jose) + passlib/bcrypt
Real-Time Alerts	FastAPI Native WebSocket + ConnectionManager
Weapon/Person Detection	Custom-trained YOLOv8l (Ultralytics)
Face Detection Model	MediaPipe BlazeFace (blaze_face_short_range.tflite)
Face Recognition	face_recognition library (dlib, 128-d encodings)
Wall-Climbing Detection	YOLOv8-pose (17 keypoints) + custom IoU tracker

Component	Specification / Technology
Video Processing	OpenCV (cv2)
Position Data Structure	JSON array of (X, Y) coordinates
Frontend Framework 1	Flutter (Dart)
Frontend Framework 2	ReactJS
Frontend Framework 3	Android (Kotlin / native)
Frontend Framework 4	React Native
Live Video Streaming	MJPEG over HTTP
Maps / Location Picker	Google Maps integration
Visual Studio Code	Primary code editor for backend (Python/FastAPI) and frontend (Flutter, React, React Native) development
Android Studio	Native Android application development and emulator-based testing
Figma	UI/UX design and mobile screen prototyping across all four frontend implementations
Postman	REST API endpoint testing and verification during backend development
Git & GitHub	Version control and team collaboration across all four team members
ArduPilot Mission Planner	Flight controller configuration, calibration, and MAVLink communication testing
Python (FastAPI)	Backend development language and web framework

4.2 Pseudocode

This section presents simplified pseudocode for the two central control loops: the drone's Manhattan-routing state machine on the Raspberry Pi 4, and the AI surveillance backend's real-time alert pipeline.

4.2.1 Drone Manhattan-Routing State Machine (Raspberry Pi 4 / pymavlink)

```

STATE = IDLE; current_cell = HOME_CELL; target_cell = None
LOOP forever:
    IF emergency_kill_signal: send_mavlink_disarm(); BREAK
    IF STATE == IDLE AND dispatch_command received:
        target_cell = dispatch_command.target_cell; STATE =
TAKEOFF
    IF STATE == TAKEOFF:
        send_mavlink_takeoff(altitude = 1.5m)
        STATE = HOVER
    IF STATE == HOVER AND target_cell != current_cell:
        dx = target_cell.x - current_cell.x
        dy = target_cell.y - current_cell.y
        IF dx != 0:
            align_yaw_to_x_axis()    # YAW_LEFT / YAW_RIGHT (90 deg)
            beams_needed = abs(dx); beams_counted = 0
            send_mavlink_velocity(SLOW_FORWARD_VELOCITY)  #
PITCH_FORWARD_START
            ELIF dy != 0:
                align_yaw_to_y_axis()    # YAW_LEFT / YAW_RIGHT (90 deg)
                beams_needed = abs(dy); beams_counted = 0
                send_mavlink_velocity(SLOW_FORWARD_VELOCITY)  #
PITCH_FORWARD_START
            ELSE: STATE = HOVER    # arrived
        ON beam_break_event:
            beams_counted += 1; update current_cell
            IF beams_counted == beams_needed:
                send_mavlink_velocity(0)    # PITCH_FORWARD_STOP
                STATE = HOVER
        IF communication timeout: send_mavlink_velocity(0); STATE =
FAULT

```

4.2.2 AI Surveillance Real-Time Alert Pipeline

```

FUNCTION detection_thread(camera):
    cap = cv2.VideoCapture(camera.rtsp_url)
    WHILE running:
        ret, frame = cap.read()

```

```

IF NOT ret: reconnect_with_backoff(); CONTINUE
IF frame_count % FRAME_SKIP != 0: CONTINUE
IF camera.type IN [FENCE, INSIDER]:
    persons = yolov8_pose_model(frame)
    FOR p IN tracker.update(persons):
        IF is_intruder(p.keypoints, polygon) AND
confirm_frames >= 3:
            IF p.track_id NOT IN alerted_ids:
                raise_alert('Wall Climbing', p.cell)
            weapons = yolov8l_model(frame)
            FOR w IN weapons WHERE conf > 0.45:
                holder = find_nearby_face(w.bbox, frame)
                IF holder AND match(holder)=='Guard': log_only()
                ELSE: raise_alert('Weapon Detected',
stray=(holder==None))
            IF camera.type == ENTRANCE:
                FOR face IN blazeface_model(frame):
                    enc = face_recognition.encode(face)
                    IF match(enc, threshold=0.50):
log_event(authorized=True)
                        ELSE: raise_alert('Unauthorized Person at Entrance')

FUNCTION raise_alert(type, ...):
    IF person_tracker.in_cooldown(person, 60s): RETURN
    alert = AlertService.create(property_id, type, severity,
...)
    db.commit(); property.active_alerts += 1
    websocket_manager.broadcast(property_id, alert.to_json())
    IF alert.severity == CRITICAL:
        drone_dispatch.dispatch(target_cell=alert.detected_cell)

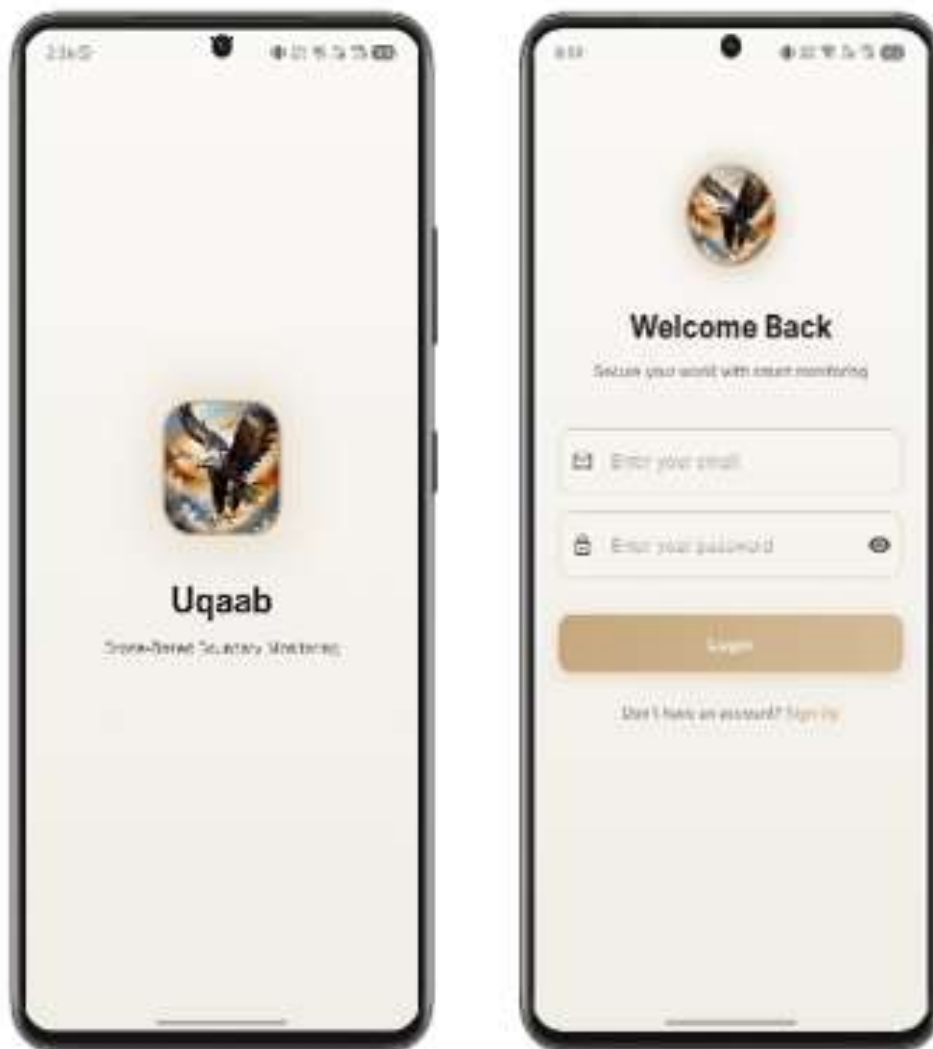
```

4.3 Application Screens (GUI)

The mobile application provides a unified interface for configuring properties, cameras, drones, and the laser grid, as well as for viewing live feeds, managing authorized people, and responding to alerts. The same set of screens has been implemented across all four platform variants developed by the team.

Splash Screen and Login Screen

The splash screen displays the project logo and application name, giving the user their first visual impression of the system. The login screen allows registered users to authenticate using their email and password via JWT-based credentials to securely access the monitoring platform.



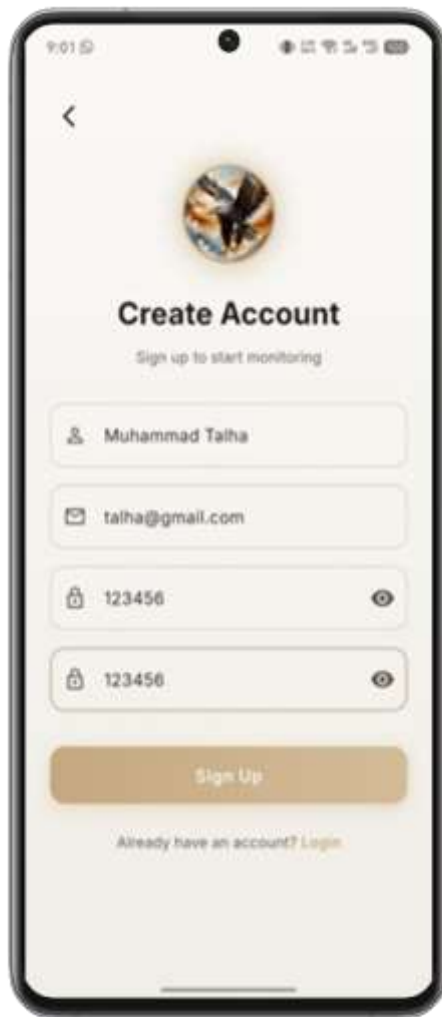
(a)

(b)

Figure 4.1: Splash Screen (a) Login Screen (b)

Sign Up Screen and Property List Screen

The Sign Up screen allows new users to create an account by entering their full name, email address, and password. Upon successful registration, the user is directed to the Property List screen, which displays all properties registered under their account along with their current camera and drone status.



(a)

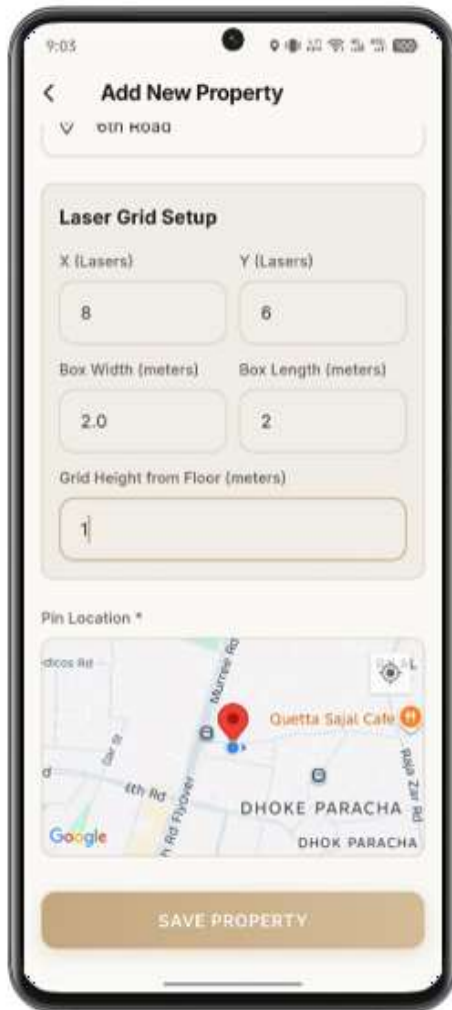


(b)

Figure 4.2: Sign Up Screen and Property List Screen

Add New Property Screen and Property List Screen

The Add New Property screen allows the user to configure the laser grid parameters for their property, including the number of X and Y lasers, box width, box length, and grid height from the floor. The user also pins the property location on an integrated map. Upon saving, the property appears in the Property List screen with its camera and drone status.



(a)



(b)

Figure 4.3: Add New Property Screen (a) Property List Screen (b)

Dashboard Screen

The Dashboard is the central monitoring interface of the application. It displays the most recent alert banner, drone status, total and online camera count, active alert count, live drone FPV feed, and live camera feeds for all cameras registered to the property, all updated in real time via WebSocket.

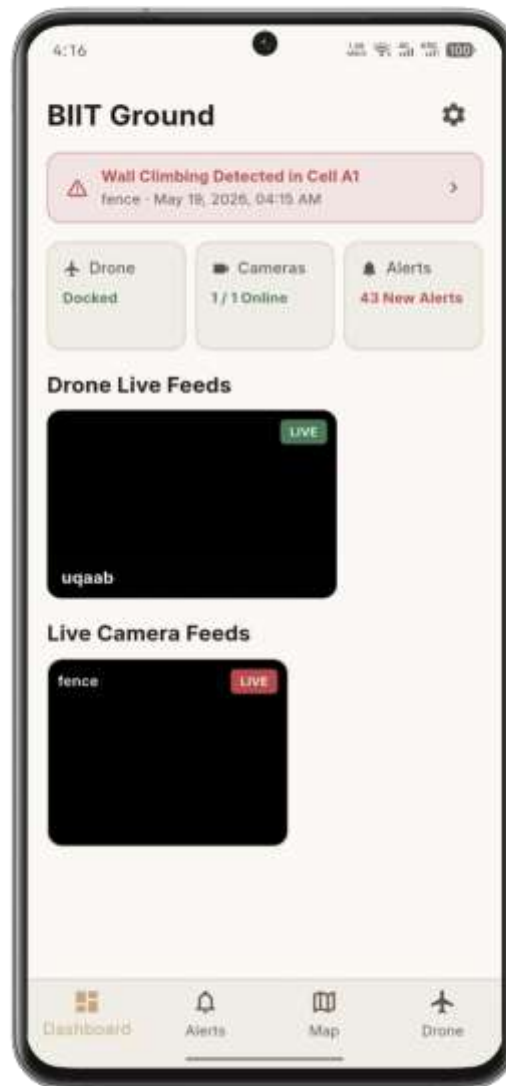


Figure 4.4: Dashboard Screen

Add New Camera Screen

The Add New Camera screen allows the user to register a new camera by providing a camera name, stream URL, zone type selection (Entrance, Fence, or Insider), and laser-grid cell assignment. Each zone type activates a different AI detection pipeline on the backend upon saving.

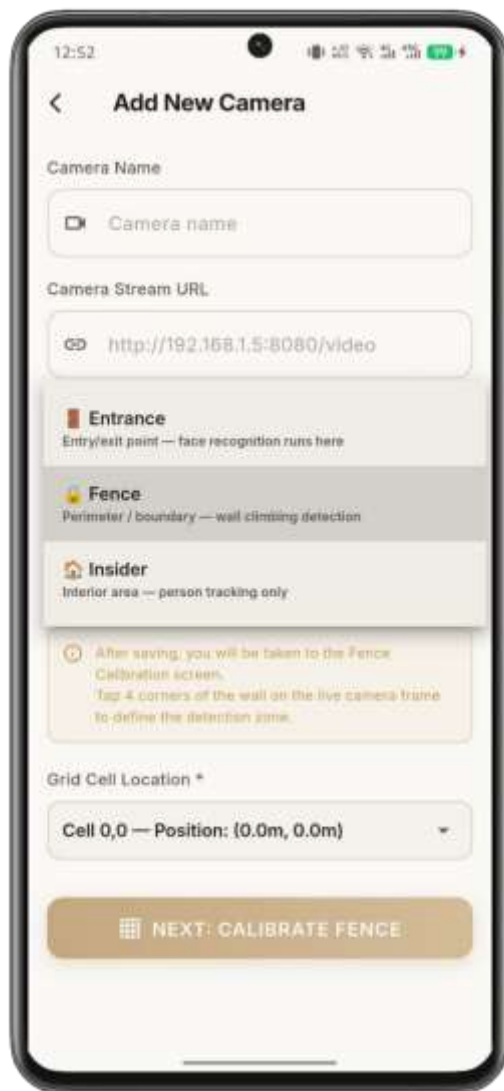


Figure 4.5: Manage Cameras Screen

Add New Drone Screen

The Add New Drone screen allows the user to register a drone by entering its name, MAVLink connection string, and home grid-cell location. The home cell defines the drone's starting position on the physical laser grid and is used as the return reference point for all patrol missions.

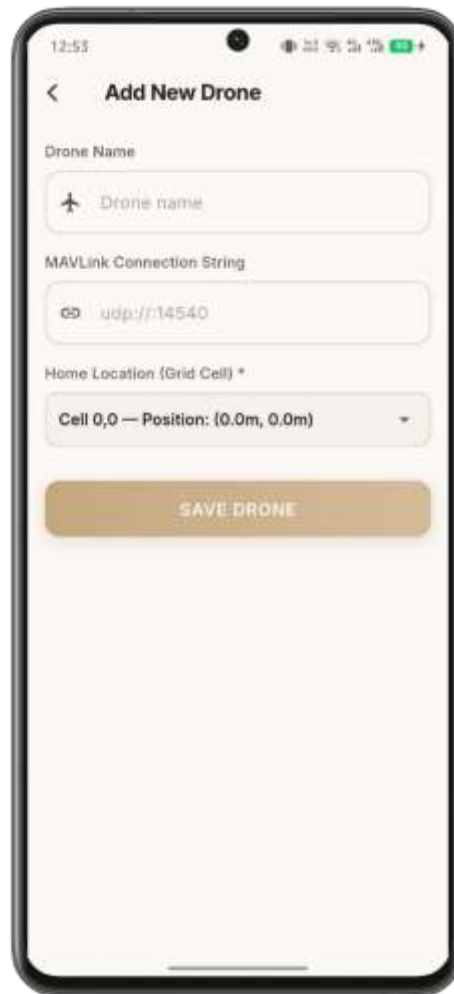
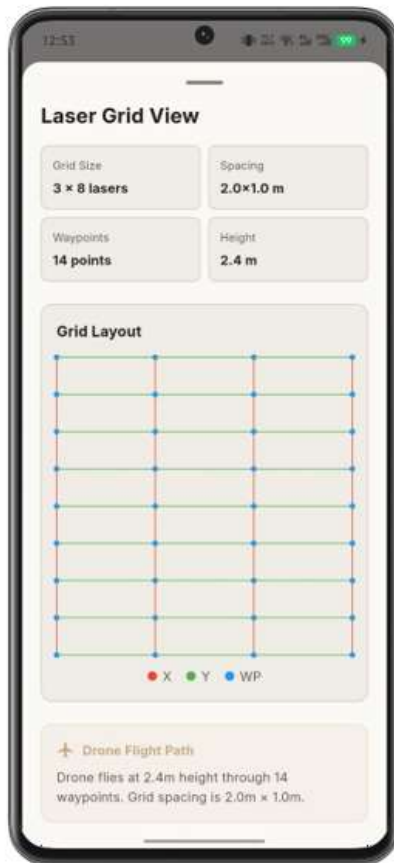


Figure 4.6: Manage Drones Screen

Laser Grid View and Grid Map Screen

The Laser Grid View screen displays the technical configuration of the property's laser grid, including grid size (3×8 lasers), beam spacing (2.0×1.0 m), total waypoint count (14 points), and operating height (2.4 m). A visual grid layout shows the X-axis lasers, Y-axis lasers, and waypoint intersections, along with a summary of the drone's flight path through the configured waypoints. The accompanying Grid Map screen presents an interactive top-down view of the same grid, marking the live positions of registered cameras and drones across each cell, with a legend distinguishing Camera, Drone, and Empty cells, alongside a list of all registered drones and cameras with their assigned home/grid-cell locations.



(a)



(b)

Figure 4.7: Laser Grid (a) Grid Map Screen (b)

Wall Calibration Screen

The Wall Calibration screen is used to define the boundary detection zone for a Fence camera. The user taps four corners directly on the live camera frame to draw the boundary polygon. A two-step progress indicator (Draw Polygon → Define Cells) guides the user through the calibration process, after which the polygon is saved to the backend for AI-based intrusion detection.



Figure 4.8: Fence Calibration Screen

Camera Cell Definition Screen

The Camera Cell Definition screen is the second step of the fence calibration process. After the boundary polygon is drawn, the user subdivides the monitored area into named detection cells such as A1 and A2. Each cell is color-coded on the live frame and saved to the backend, enabling the system to report the precise cell location of any detected intrusion in its alerts.

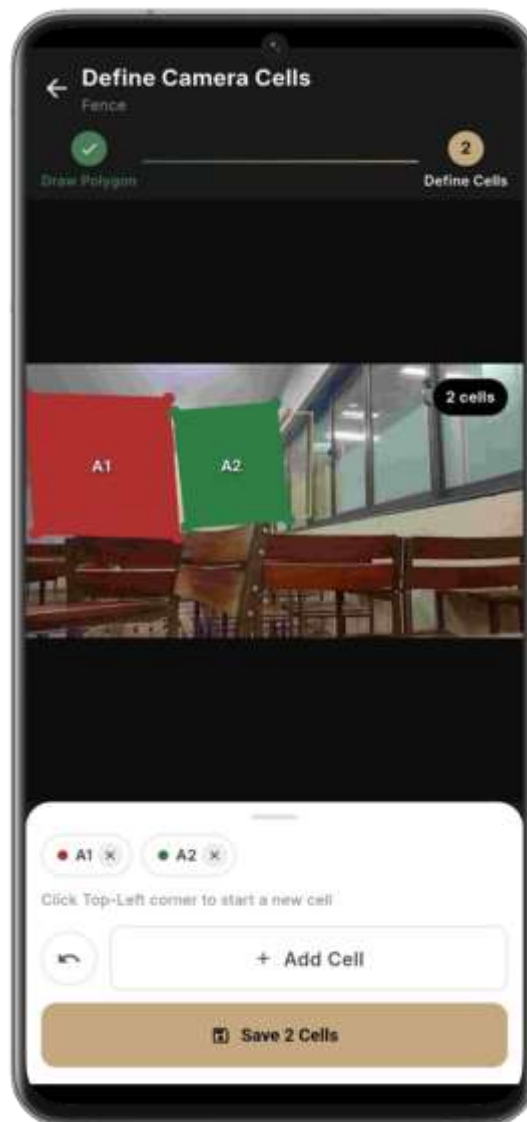


Figure 4.9: cell Calibration Screen

Alerts Screen

The Alerts screen presents a chronological list of all detected security events for the selected property. Each alert card displays a thumbnail snapshot, event type, camera name, timestamp, and confidence percentage. The list can be filtered by New or All alerts, allowing the property owner to quickly identify and respond to unreviewed events.



Figure 4.10: Alerts Screen

Alert Detail Screen

The Alert Detail screen provides the complete information for a selected alert, including the event snapshot image, camera name, grid position, detected cell, timestamp, confidence percentage, severity level, and current status. The screen provides four action buttons — View Live Camera, Dispatch Drone, False Positive, and Resolve — allowing the operator to take immediate action directly from the alert.



Figure 4.11: Alert Detail Screen

Add Authorized Person Screen

The Add Authorized Person screen allows the administrator to enroll an individual into the face recognition system. The user enters the person's full name, selects their role (Guard, Guest, or Authorized Person), and uploads three face photos from different angles (Front, Left, Right). Tapping the Enroll Face button triggers the backend to generate a 128-dimensional face encoding, after which the individual will be recognized at all entrance cameras and excluded from unauthorized-person alerts.

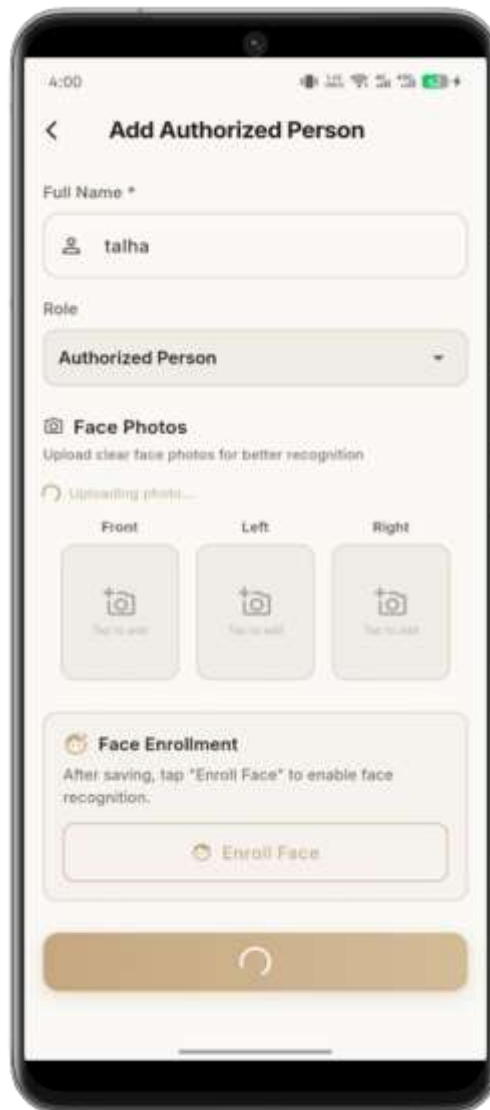


Figure 4.12: Authorized People Screen

Drone Control Screen

The Drone Control screen gives the operator direct control over the registered drone. It displays a live FPV video feed from the drone's onboard camera, a real-time laser grid map showing the drone's current cell position highlighted in orange, and RTH (Return to Home) and LAND control buttons. A directional control pad at the bottom allows the operator to manually navigate the drone to any grid cell when needed.



Figure 4.13: Drone Control Screen

Real-Time Alert Notifications

The system delivers real-time push notifications directly to the property owner's mobile device lock screen the moment a critical event is detected. Each notification displays the alert type and camera name, ensuring the owner is instantly informed of any boundary intrusion even when the application is running in the background.



Figure 4.14: Alert Notifications on Mobile

CHAPTER 5

CONCLUSION

5.1 Concluding Remarks

The Drone Based House Boundary Wall Monitoring System has been successfully designed and implemented as a complete, end-to-end autonomous perimeter security platform, demonstrating that the three most persistent challenges in residential boundary security — passive surveillance, GPS-dependent drone navigation, and disconnected detection-response pipelines — can be solved simultaneously within a single, low-cost, integrated architecture. The custom-trained YOLOv8l model, MediaPipe BlazeFace with face recognition, and YOLOv8-Pose with a custom IoU-based tracker operate concurrently across Entrance, Fence, and Insider cameras, providing comprehensive real-time threat classification across all three boundary security vectors: unauthorized entry, perimeter climbing, and on-site weapon presence. The laser-grid positioning system — built from KY-008 laser modules, five ESP32 microcontrollers communicating via ESP-NOW and USB serial, and a Manhattan-style routing state machine executed via pymavlink on a Raspberry Pi 4 — conclusively proves that precise, deterministic drone navigation is achievable in a GPS-denied environment without expensive vision-based localization hardware. The deliberate separation of the flight controller and companion computer power supplies via independent LM2596 regulators demonstrates that careful hardware-level design decisions are as critical to system reliability as the software architecture itself. The unified FastAPI backend, PostgreSQL database with thirteen normalized tables, and real-time WebSocket alerting pipeline bind all subsystems into a single coherent platform capable of multi-property, multi-camera, and multi-drone operation, while the four parallel mobile applications — Flutter, ReactJS, native Android, and React Native — ensure the system is accessible to property owners on every major platform. Most significantly, the project achieves what no existing residential security product currently offers: the moment a threat is detected at a specific grid cell, the system does not merely log it or notify the owner — it physically dispatches the drone to that exact location for immediate visual verification, closing the loop between detection and response autonomously, precisely, and without human intervention.

5.2 Future Direction

- Expanding the laser grid beyond 5×7 cells to cover larger properties, and exploring modular grid sections that can be combined for irregularly shaped boundaries.
- Implementing automated recovery logic for grid-tracking failures (e.g., re-localization via a known reference cell) to reduce reliance on manual failsafe intervention.
- Extending the custom YOLOv8l dataset with additional weapon classes and environmental conditions (night-time, adverse weather) to improve detection robustness.
- Adding support for multiple simultaneous drones, with the dispatch backend selecting the nearest available drone to a detected alert's grid cell.
- Performing a unified usability study across the four frontend implementations to converge on a single primary platform for production deployment.
- Exploring battery-swap or charging-dock automation so the drone can return to its home cell, recharge, and resume patrol autonomously.
- Migrating lightweight detection models (YOLOv8n variants) to the Raspberry Pi 4 for fully offline patrol, reducing dependency on the backend laptop.

5.3 Limitations

- The drone's positioning accuracy is fundamentally limited to the resolution of the laser grid (individual cell size); sub-cell positioning is not possible.
- The system relies on a manual failsafe for grid-tracking failures; there is currently no automated recovery from a missed or unexpected beam-break sequence.
- Yaw correction depends on a manual RC-based calibration step prior to autonomous operation, introducing a setup dependency before each flight session.
- The communication pipeline (ESP-NOW $\rightarrow$ USB Serial $\rightarrow$ Wi-Fi) introduces cumulative latency, requiring the drone to operate at reduced forward velocity, which limits patrol speed.
- AI detection accuracy is dependent on camera placement, lighting conditions, and the size and diversity of the custom-annotated training dataset.

- The laser grid and drone hardware are not weatherproofed for operation in rain or extreme temperatures.
- A single 5200mAh 4S LiPo provides approximately 10–15 minutes of patrol time; longer perimeters require battery swap or additional battery capacity.

REFERENCES

- ArduPilot Development Team. (2023). *pymavlink: Python MAVLink interface and utilities* [Computer software]. Retrieved from <https://github.com/ArduPilot/pymavlink>
- ArduPilot Development Team. (2024). *MAVLink Developer Guide*. Retrieved from <https://ardupilot.org/dev/docs/mavlink-basics.html>
- Espressif Systems. (2023). *ESP-NOW Protocol Documentation: ESP-IDF Programming Guide*. Retrieved from https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html
- Geitgey, A. (2017). *face_recognition: The world's simplest facial recognition API for Python and the command line* [Computer software]. Retrieved from https://github.com/ageitgey/face_recognition
- Guna, R., Ganeriwala, S., & Bhattacharyya, A. (2024). Machine learning engineering for object detection and tracking by drone-captured videos. *arXiv preprint arXiv:2410.18230*. Retrieved from <https://arxiv.org/abs/2410.18230>
- Jocher, G., Chaurasia, A., & Qiu, J. (2023). *YOLO by Ultralytics* (Version 8.0.0) [Computer software]. Retrieved from <https://github.com/ultralytics/ultralytics>
- Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., & Grundmann, M. (2019). MediaPipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*. Retrieved from <https://arxiv.org/abs/1906.08172>
- OpenCV Team. (2023). *OpenCV: Open source computer vision library*. Retrieved from <https://opencv.org/>
- PostgreSQL Global Development Group. (2023). *PostgreSQL: The world's most advanced open source relational database*. Retrieved from <https://www.postgresql.org/>
- Ramirez, S. (2023). *FastAPI: A modern, fast web framework for building APIs with Python* [Computer software]. Retrieved from <https://fastapi.tiangolo.com/>
- ST Microelectronics. (2020). *VL53L1X time-of-flight ranging sensor datasheet*. Retrieved from <https://www.st.com/resource/en/datasheet/vl53l1x.pdf>

ANNEXURE

Annexure A: System Initialization

The following procedure brings the Drone Based House Boundary Wall Monitoring System to a fully operational state:

1. Power on the laser grid TX side (12V 3A adapter) and verify all KY-008 laser transmitters illuminate.
2. Power on the laser grid RX side (12V 3A adapter) and confirm all four Slave ESP32 units and the Master ESP32 boot successfully and establish ESP-NOW communication links.
3. Verify laser alignment — every receiver module LED should be ON, confirming all beams are unbroken.
4. Connect the Master ESP32 to the Backend Laptop via USB Serial cable.
5. Start the FastAPI surveillance backend (uvicorn), which automatically initializes detection threads for all configured Entrance, Fence, and Insider cameras.
6. Start the FastAPI drone navigation backend, which begins listening for beam-break events over the USB serial connection.
7. Power on the drone (5200mAh 4S LiPo) and confirm the Pixhawk 6C Mini and Raspberry Pi 4 boot independently via their respective LM2596 regulators.
8. Confirm the Raspberry Pi 4 connects to the local Wi-Fi router and establishes a link with the backend laptop.
9. Open the mobile application, log in, and confirm the dashboard reports the expected number of cameras online and the drone status as Docked or Ready.

Annexure B: Mission Execution

The following procedure describes a standard manual dispatch and patrol mission:

1. In the mobile application, navigate to the Drone Control screen and select the target drone.

2. Confirm the drone's home grid cell matches its physical starting position on the laser grid.
3. Set the target cell using the Grid Map / Laser Grid View screen.
4. Issue the dispatch command. The drone transitions through TAKEOFF → HOVER → (YAW_LEFT / YAW_RIGHT + PITCH_FORWARD_START / PITCH_FORWARD_STOP, repeated per Manhattan routing steps) → HOVER at the target cell.
5. Monitor the live FPV feed and real-time grid position indicator on the Drone Control screen throughout the mission.
6. Upon mission completion, issue LAND or RETURN_TO_HOME as required.

Annexure C: Alert-Triggered Drone Dispatch

The following procedure describes the automated dispatch flow triggered by a critical AI surveillance alert:

1. A detection thread on a Fence or Insider camera identifies a critical event — such as wall climbing or weapon detection — and determines the detected_cell coordinate.
2. AlertService creates the alert record in the database and broadcasts it via WebSocket. The mobile application immediately displays the alert with a Dispatch Drone action button.
3. The operator, or an automated critical-severity rule, triggers dispatch. The alert's detected_cell is passed to the drone navigation backend as the target cell.
4. The drone autonomously executes the Manhattan-routing sequence to the target cell, counting beam breaks to confirm each movement step.
5. Upon arrival, the drone hovers at the target cell and its live FPV feed is displayed alongside the original alert snapshot for immediate visual verification.
6. The operator reviews the FPV feed and marks the alert as Resolved or False Positive accordingly.

Annexure D: Emergency Protocols

The following manual failsafe procedures are available to the operator at any point during system operation:

EMERGENCY KILL: Immediately cuts all motor output and forces disarm regardless of the drone's current altitude or state. Accessible from any screen in the mobile application and via the designated RC transmitter switch. To be used only when immediate motor stoppage is necessary and a controlled landing is not possible.

EMERGENCY LAND: Commands an immediate controlled vertical descent to ground level followed by automatic disarm. This is the preferred emergency procedure when the drone is airborne, as it ensures a safe, controlled landing rather than an uncontrolled fall.

FAULT State Recovery: If the drone enters the FAULT state due to a communication pipeline timeout or an unexpected beam-break sequence, it will automatically hold its current position in HOVER. The operator should visually confirm the drone's physical location, then issue either a corrected target cell command, RETURN_TO_HOME, or EMERGENCY LAND as appropriate to the situation.

Manual RC Override: At all times, the HotRC HT6A RC transmitter retains full authority to override any autonomous command. The operator may apply yaw correction at any point during autonomous flight, or switch to full manual control by toggling the flight mode switch to Stabilize or Manual mode on the transmitter.